ÉCOLE POLYTECHNIQUE DE THIÈS · CRI / NOC

Trans-DIC1
Dossier d'architecture complet

Plateforme de supervision des équipements réseau et serveurs

Pourquoi, comment, donc : réalisations, schémas, décisions, écarts et feuille de route

Version du 28 octobre 2026 · Branche develop
Contenu : dossier d'architecture, six ADR, catalogue des diagrammes, correspondance du modèle persistant

Trans-DIC1 — Dossier d'architecture complet

Date : 28 octobre 2026 · Branche : `develop` · Auteur du dépôt : Boubacar Sidibe

Ce document répond à trois questions pour chaque élément du projet : **pourquoi** il existe, **comment** il est réalisé, et **donc** ce qui en découle. Il distingue toujours ce qui est **réalisé dans le code** de ce qui est **décidé mais pas encore codé** (cible).

Légende : ✅ réalisé et présent dans le code · 🟡 décidé (ADR ou diagramme), à implémenter · 🟣 prévu au backlog, sans décision détaillée.

Table des matières

1. Résumé
2. Le problème et les objectifs
3. Ce qui a été réalisé
4. Vue d'ensemble de l'architecture
5. Architecture du backend
6. Modèle de données
7. Comportements clés (séquences et états)
8. Sécurité
9. Frontend
10. Qualité, CI/CD et exploitation
11. Décisions d'architecture et justifications
12. Écarts entre le code et la cible, risques
13. Feuille de route
14. Annexes

---

1. Résumé

Trans-DIC1 est une plateforme interne de supervision des équipements réseau et serveurs de l'École Polytechnique de Thiès (EPT), destinée au CRI et à son centre d'opérations réseau (NOC). Elle doit inventorier les équipements, collecter leurs mesures, détecter les dépassements de seuil, gérer le cycle de vie des alertes, notifier par e-mail et produire des rapports.

Où en est le projet : le socle technique et la qualité sont solides (API sécurisée, inventaire, modèle de données complet, CI exigeante, Docker). Le cœur fonctionnel de la supervision (collecte, évaluation des seuils, alertes, notifications, rapports, temps réel) est **modélisé mais pas encore codé**.

Principaux choix : monolithe modulaire Spring Boot + React, PostgreSQL, JWT court, modèle de domaine riche vérifié par ArchUnit. Six ADR documentent les décisions. Les six sont acceptées ou proposées ; les quatre dernières (0003 à 0006) ont été rédigées après une revue critique appuyée sur des recherches (section

11).

---

2. Le problème et les objectifs

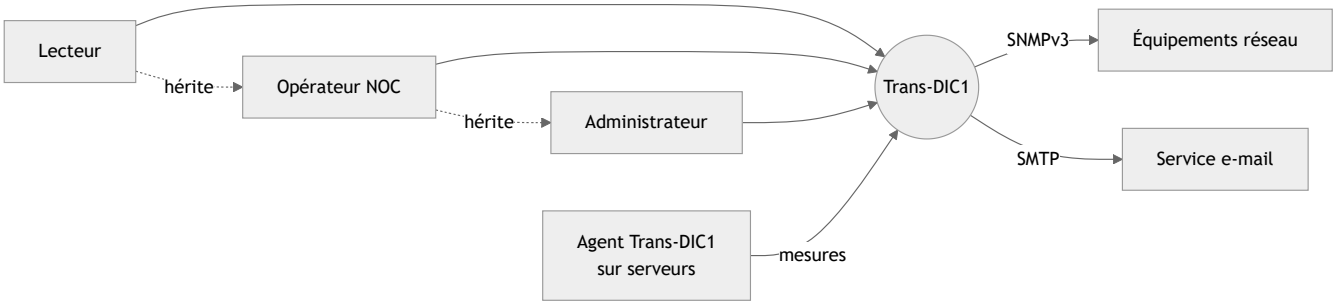
Pourquoi. Le CRI doit savoir en continu quels équipements du campus fonctionnent, être averti avant les utilisateurs et garder une trace des incidents. Sans outil, la détection dépend des plaintes.

Acteurs (trois rôles humains, hiérarchiques) :

Rôle	Peut
Lecteur (`VIEWER`)	Consulter tableau de bord, équipements, métriques, alertes, rapports
NOC (`NOC_OPERATOR`)	Tout ce que fait le lecteur + gérer l'inventaire, prendre en charge et résoudre les alertes
Administrateur (`ADMIN`)	Tout + utilisateurs, rôles, seuils, assignation des alertes

Systèmes externes : agent Trans-DIC1 installé sur les serveurs, équipements SNMP, service e-mail.

Exigences structurantes (issues de l'ADR-0001) : fonctionner en réseau interne sans Internet, ne jamais exposer la topologie, ne jamais mettre de secret dans Git ni dans les journaux, tracer les actions sensibles, builds reproductibles.



Lecture : la flèche pointillée traduit la hiérarchie des rôles (Lecteur ⊂ NOC ⊂ Administrateur).

---

3. Ce qui a été réalisé

Source : historique git (`develop` , 41 commits d'avance sur `main`), code et documentation vérifiés dans le dépôt.

3.1 Tableau d'avancement

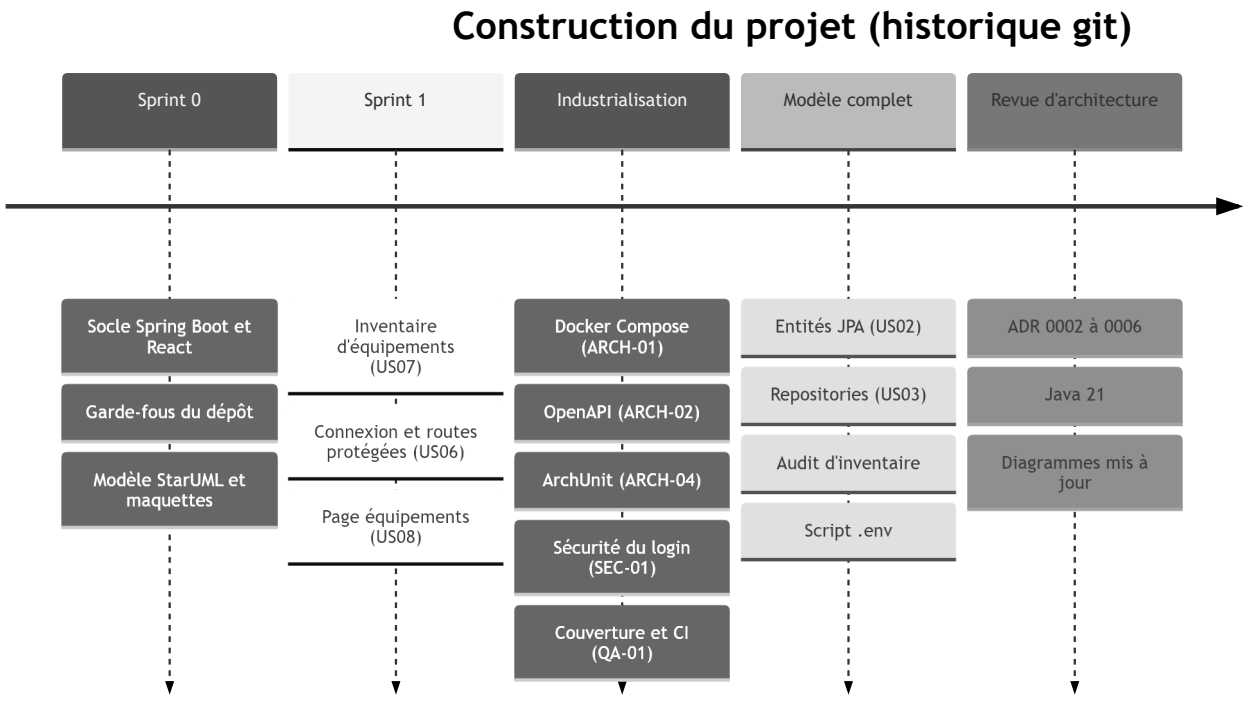
Domaine	Élément	État	Preuve dans le dépôt
Fondations	Monorepo `backend/` + `frontend/` , Maven Wrapper, Node 24	✅	`README.md` , `pom.xml`

Domaine	Élément	État	Preuve dans le dépôt
Fondations	Docker Compose (PostgreSQL 16, Mailpit, backend, frontend) + script `.env`	✅	`docker-compose.yml`, `scripts/init-dev-env.ps1`
Authentification	Login JWT, hash de mot de passe, réponse uniforme 401, compte factice anti-énumération	✅	`AuthService`, `JwtService`
Authentification	Limitation de débit par IP (Bucket4j) + verrouillage progressif du compte (SEC-01)	✅	`LoginRateLimitFilter`, V6
Authentification	Renouvellement de jeton par cookie HttpOnly avec rotation, déconnexion	✅	`AuthController`, `AuthService`, `RefreshTokenTests`
Authentification	Réinitialisation de mot de passe	🟣	table `password_reset_token` créée
Autorisation	Rôles `ADMIN` / `NOC_OPERATOR` / `VIEWER` sur les endpoints (`@PreAuthorize`)	✅	`EquipmentController`
Inventaire	CRUD équipements `/api/v1/equipements`, DTO validés, erreurs `problem+json`	✅	`EquipmentController`, `ApiExceptionHandler`
Inventaire	Audit des mutations d'inventaire (US07)	✅	`AuditService`, `EquipmentAuditTests`
Métriques	Lecture des dernières mesures `/api/v1/metrics/latest`, mesure CPU synthétique	✅	`MetricController`, V4
Métriques	Partitions mensuelles, agrégats horaires, purge sans perte d'historique	✅	V9, V12, `MetricRetentionService`, `MetricRetentionTests`
Modèle	28 entités JPA, 28 tables, contraintes `CHECK`, index uniques partiels (US02)	✅	V7, V8, `PersistenceModelTests`
Modèle	Repositories du domaine et leurs tests (US03)	✅	`*Repository`
API	Contrat OpenAPI + Swagger UI (désactivés en prod), types TypeScript générés	✅	`OpenApiConfig`, `docs/api/openapi.yaml`
Observabilité	Identifiant de corrélation dans chaque journal et réponse, `/actuator/health` sans détails	✅	`CorrelationIdFilter`
Frontend	Connexion, routes protégées, tableau de bord, page de gestion des équipements (US06, US08)	✅	`frontend/src/`
Qualité	Règles ArchUnit (18 contrôles), JaCoCo ≥ 80 % lignes / 60 % branches, Spotless, Vitest ≥ 75 %, ESLint	✅	`pom.xml`, `vite.config.js`
CI	Hygiène du dépôt, backend, frontend, gitleaks, Trivy, actions épinglées par empreinte	✅	`repository-guard.yml`
Documentation	6 ADR, ~40 diagrammes Mermaid, modèle StarUML, maquettes, 2 présentations	✅	`docs/`, `deliverables/`

Domaine	Élément	État	Preuve dans le dépôt
Collecte	Enregistrement des agents (jeton), réception des mesures `POST /api/v1/agent/measures`, stockage et évaluation	✅	`AgentController`, `MeasureIngestionService`, `AgentApiTests`
Collecte	Planificateur et adaptateur SNMP4J	🟡	ADR-0006, diagrammes
Alertes	Évaluation des seuils : confirmation, hystérésis, déduplication, réouverture, maintenance	✅	`AlertEvaluationService`, `AlertEvaluationTests`
Alertes	Prise en charge, assignation, résolution manuelle, archivage, API et écrans	🟣	modèle prêt, pas d'API
Notifications / rapports	Envoi e-mail, tentatives, rapports planifiés	🟣	entités créées, aucun service
Temps réel	Flux SSE vers le tableau de bord	🟡	ADR-0006
Déploiement	Nginx, TLS, staging, production	🟣	prérequis externes ouverts

Vérification faite pour ce dossier : `./mvnw verify` sous Java 21 se termine par `BUILD SUCCESS` avec 157 tests backend sans échec (PostgreSQL 16 embarqué, sans Docker), seuils JaCoCo et Spotless passés. Côté frontend : 20 tests, couverture 91 %, lint sans erreur, build et audit des dépendances sans faille. La CI distante n'a pas été relancée.

3.2 Chronologie



3.3 Ce que cela signifie

Donc : le projet a terminé de poser les fondations et avance sur le premier flux métier de bout en bout (se connecter, gérer l'inventaire, voir une mesure). Tout le reste de la valeur de supervision reste à construire. C'est cohérent avec le backlog (Sprint 2 : collecte, tableau de bord, métriques).

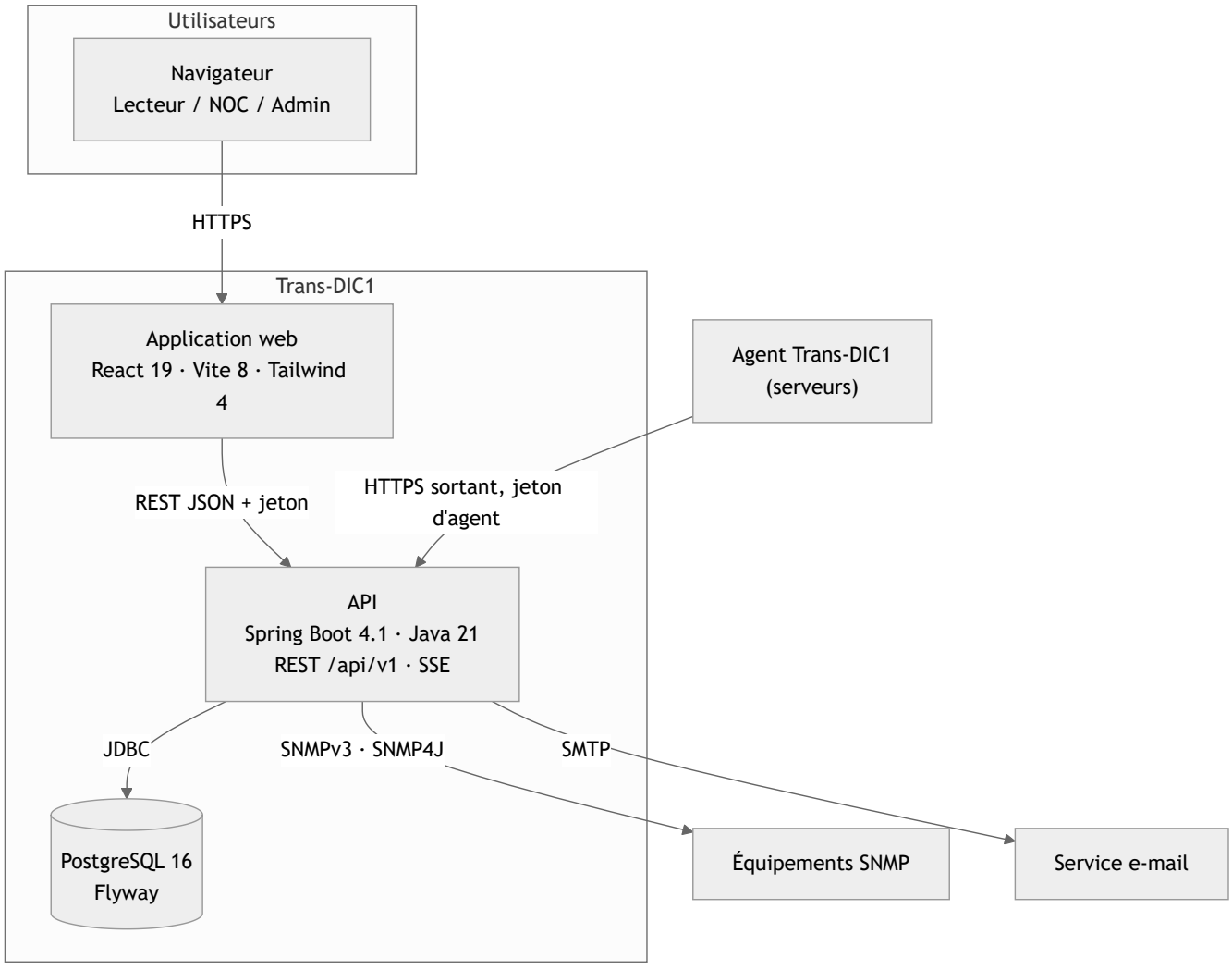
---

4. Vue d'ensemble de l'architecture

4.1 Contexte et conteneurs

Pourquoi un monolithe modulaire : deux développeurs, un serveur, un domaine très lié (une mesure déclenche une alerte qui déclenche une notification). Des microservices ajouteraient du réseau, du déploiement et de la complexité sans bénéfice. Un monolithe découpé en modules avec règles vérifiées garde la possibilité d'extraire un module plus tard.

Comment : un backend Spring Boot unique, un frontend React servi séparément, une base PostgreSQL. La collecte vit d'abord dans le backend (ADR-0006) et pourra devenir un processus à part si la charge l'exige.



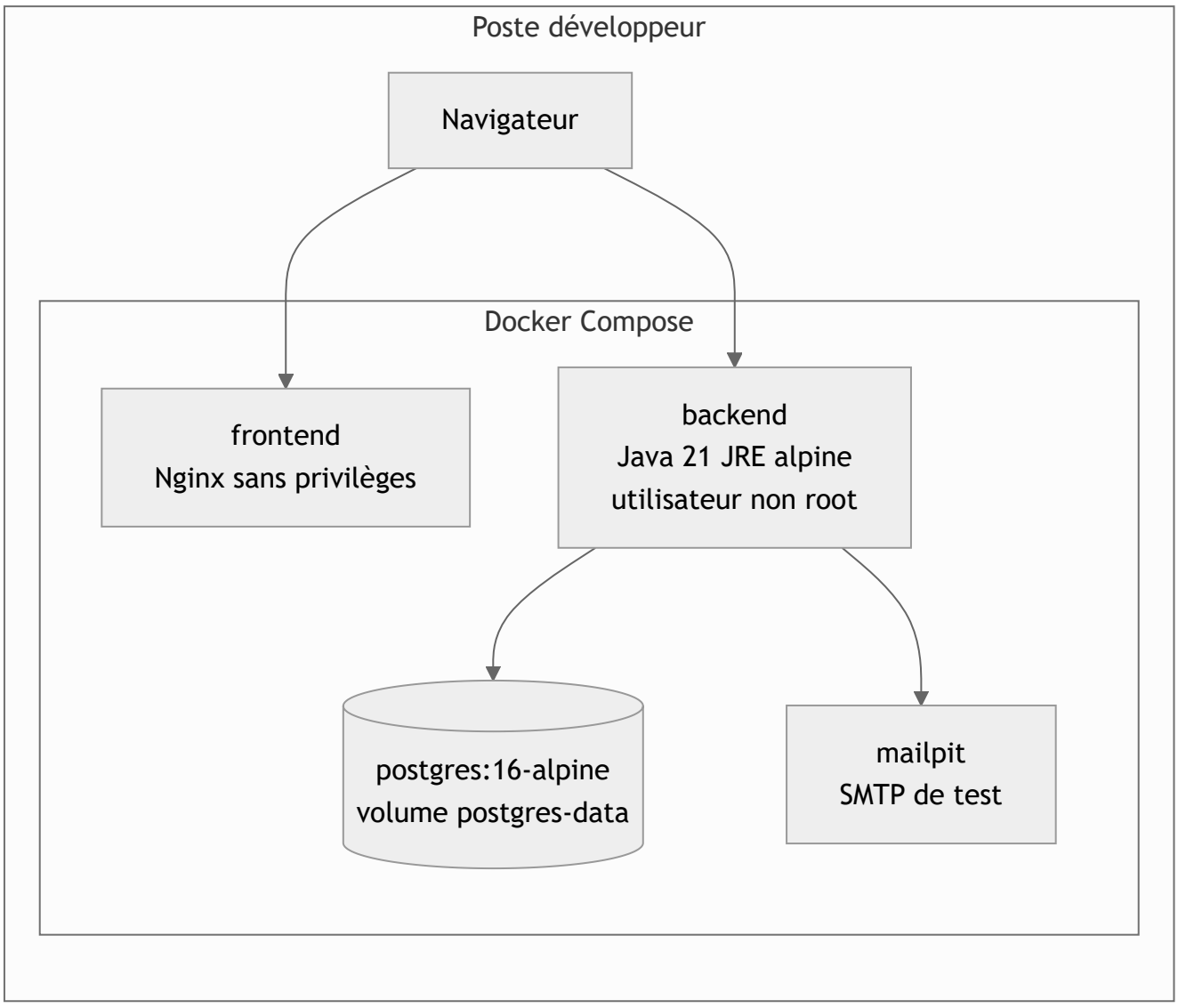
Réalisé : navigateur → web → API → base. Cible : agent, SNMP, SMTP.

Donc : le seul flux entrant vers le centre est HTTPS. Les pare-feu des sites n'ont qu'à autoriser la sortie de l'agent vers l'API.

4.2 Déploiement

Pourquoi : les builds doivent être identiques entre développement, staging et production (ADR-0001).

Comment (réalisé) : `docker compose up --build` lance quatre conteneurs. PostgreSQL est exposé sur `127.0.0.1:5433` seulement. Les secrets viennent d'un `.env` généré par `scripts/init-dev-env.ps1` et jamais commité.



Cible (🟡 / 🟣) : un Nginx en frontal unique, qui sert l'application web et relaie l'API sur le même site (nécessaire pour les cookies `SameSite=Strict` de l'ADR-0005), TLS, serveur de données séparé, staging puis production. L'hébergement EPT n'est pas encore connu.

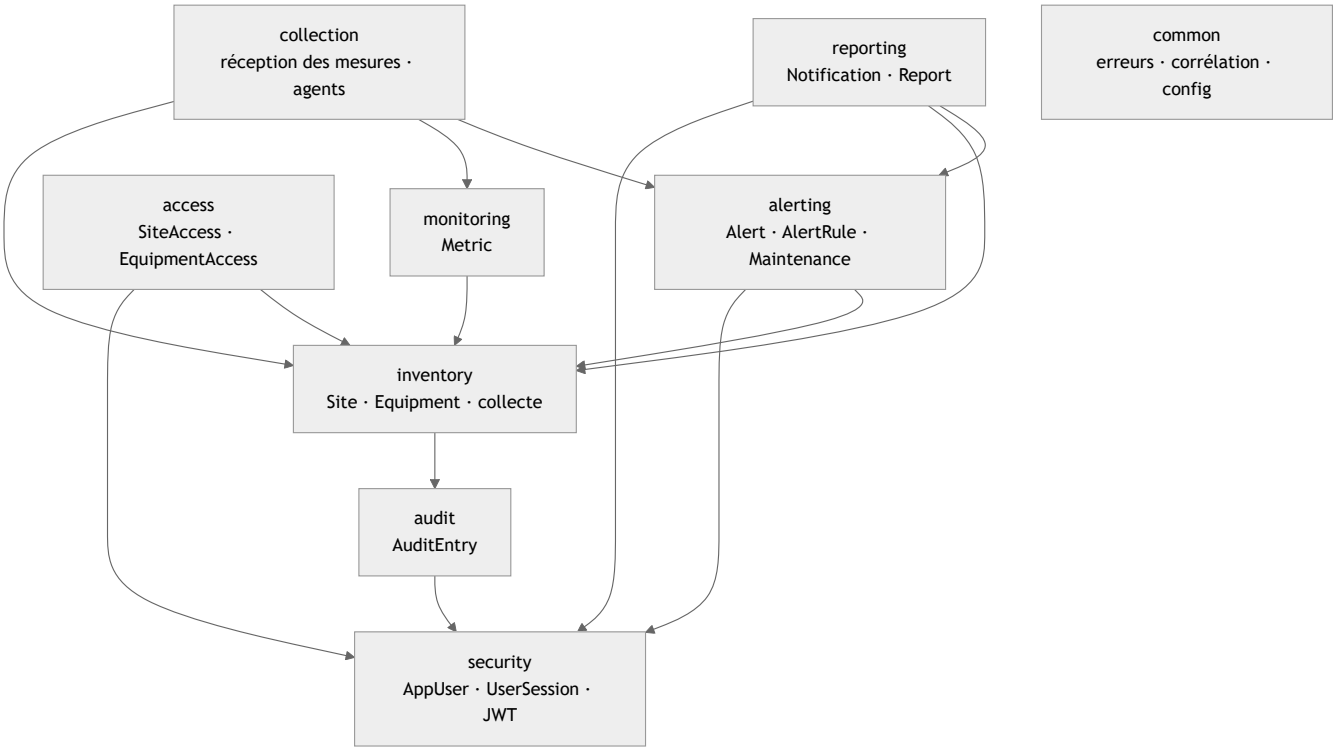
---

5. Architecture du backend

5.1 Modules

Pourquoi : isoler les responsabilités pour qu'un changement dans les rapports ne casse pas la sécurité.

Comment : un paquet Java par module sous `sn.ept.transdic1`, chacun découpé en `controller`, `service`, `repository`, `domain`, `dto`. Les dépendances autorisées sont codées dans `ArchitectureTests` et vérifiées à chaque build.



Donc : `access` est un module à part pour éviter un cycle `security` ↔ `inventory`. `collection` orchestre : elle reçoit les mesures, les stocke via `monitoring` et les fait évaluer par `alerting`, que personne d'autre ne rappelle (aucun cycle). Le moteur d'alertes lit l'historique par un port (`MetricHistory`) que `collection` implémente. Les modules du bas (`security`, `audit`, `inventory`) ne connaissent jamais `alerting` ni `reporting`.

5.2 Règles automatiques (ArchUnit)

Règle	Pourquoi
Contrôleur → service → repository, jamais autrement	La logique et les transactions restent dans les services
Le domaine ne dépend ni de l'API ni de la persistance	Le modèle reste pur
Aucun cycle entre modules, sens imposé	Modules modifiables séparément
Les contrôleurs n'exposent aucune entité	L'API échange des DTO, jamais des entités JPA
`Equipment` sans sous-classe	Serveur, routeur… sont des valeurs de `EquipmentType`

Règle	Pourquoi
Pas d'injection par champ	Dépendances explicites et testables

Donc : l'architecture est un contrat exécutable et non un document qui vieillit.

5.3 Endpoints réalisés

Méthode et chemin	Rôle requis	Description
`POST /api/v1/auth/login`	public (limité en débit)	Authentifie et renvoie un JWT
`GET /api/v1/equipements` · `GET /{id}`	authentifié	Liste et détail
`POST /api/v1/equipements`	`ADMIN`, `NOC_OPERATOR`	Création (201 + `Location`)
`PUT /api/v1/equipements/{id}`	`ADMIN`, `NOC_OPERATOR`	Modification
`DELETE /api/v1/equipements/{id}`	`ADMIN`, `NOC_OPERATOR`	Suppression
`GET /api/v1/metrics/latest`	authentifié	Dernières mesures
`POST /api/v1/auth/refresh` · `/logout`	cookie de renouvellement	Renouvelle le jeton d'accès (rotation) · révoque la connexion
`POST` · `DELETE /api/v1/equipements/{id}/agent`	`ADMIN`	Enregistre un agent (jeton affiché une fois) · le révoque
`POST /api/v1/agent/measures`	jeton d'agent	Reçoit un lot de mesures (500 au plus), les stocke et évalue les alertes
`GET /actuator/health`	public	Santé, sans détail

Cible (chemins recommandés, 🟡) : `/api/v1/regles-alertes`, `/api/v1/alertes/{id}/prise-en-charge`, `/resolution`, `/api/v1/rapports`, réception des mesures d'agent, flux SSE.

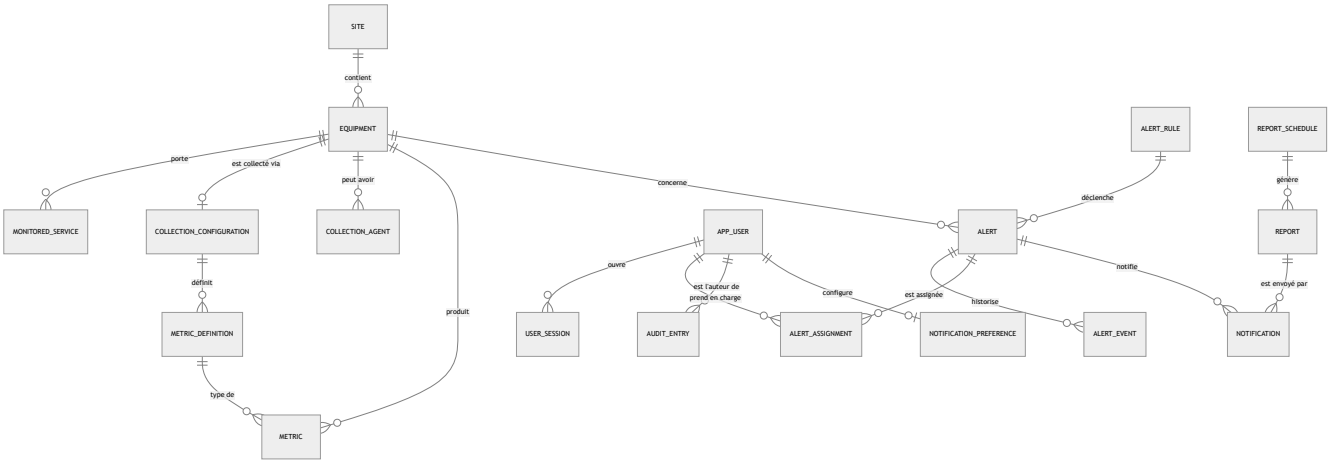
---

6. Modèle de données

6.1 Vue d'ensemble

Pourquoi : le modèle a été validé avec le métier le 6 octobre 2026 et remplace des hypothèses antérieures (états d'alerte, rôle, Zabbix). Il est écrit en français dans les diagrammes et en anglais dans le code (décision d'équipe, correspondance dans `persistence-mapping.md`).

Comment : 28 tables, identifiants UUID, énumérations stockées en texte et protégées par `CHECK`, durées en secondes, horodatages avec fuseau.



Les tables de portée (`rule_site_scope` , `rule_equipment_scope` , `schedule_*_scope`), d'accès (`site_access` , `equipment_access`), `maintenance` , `password_reset_token` et `notification_attempt` sont omises pour la lisibilité ; elles figurent dans `persistence-mapping.md` .

6.2 Règles garanties par la base

Règle métier	Garantie
Une seule alerte active par équipement et règle	Index unique partiel (PostgreSQL)
Un seul agent actif par équipement	Index unique partiel
Une notification vient d'une alerte **ou** d'un rapport	`CHECK`
Une maintenance cible un site **ou** un équipement	`CHECK`
Mode `SNMP` exige une version, mode `AGENT` n'en a pas	`CHECK`

Pourquoi dans la base : une règle codée seulement dans le service peut être contournée par une seconde instance ou un script. **Donc** un test sur un vrai PostgreSQL est indispensable (ADR-0003) : H2 ne gère pas ces index.

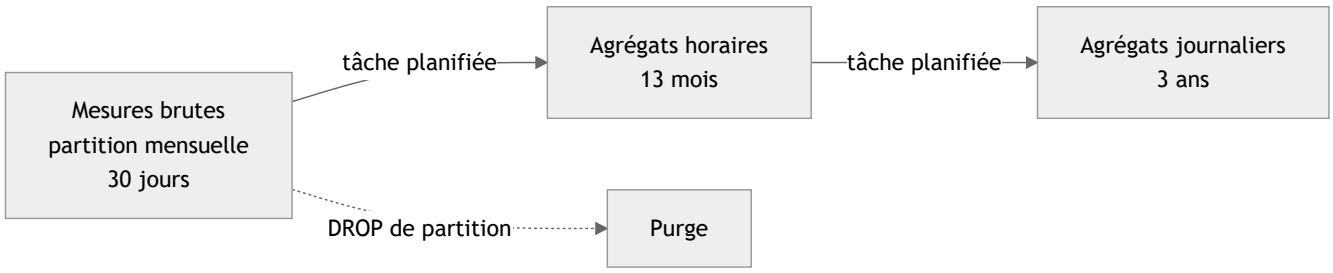
6.3 Migrations

Version	Contenu
V1	Base vide (socle)
V2	Table `equipment` et cinq équipements fictifs
V3	`site` , `app_user` , `audit_entry`
V4	`metric` (mesure CPU synthétique)
V5	Nettoyage des colonnes héritées
V6	Verrouillage de compte (tentatives, `locked_until`)
V7	Modèle persistant complet

Version	Contenu
V8	Index uniques partiels
V9	Table `metric` partitionnée par mois, clé `(collected_at, id)`, fonction `ensure_metric_partition`
V10	Sessions : `family_id`, `replaced_by_id` (rotation des jetons de renouvellement)
V11	`alert_rule.recovery_threshold` (hystérésis) avec contrainte de cohérence
V12	`metric_hourly` (agrégats horaires) et fonction `drop_expired_metric_partitions`
V13	`collection_agent.token_hash` (empreinte du jeton d'agent)

La prochaine migration est la V14. Un seul dossier de migrations.

6.4 Stockage des métriques (cible, ADR-0004)

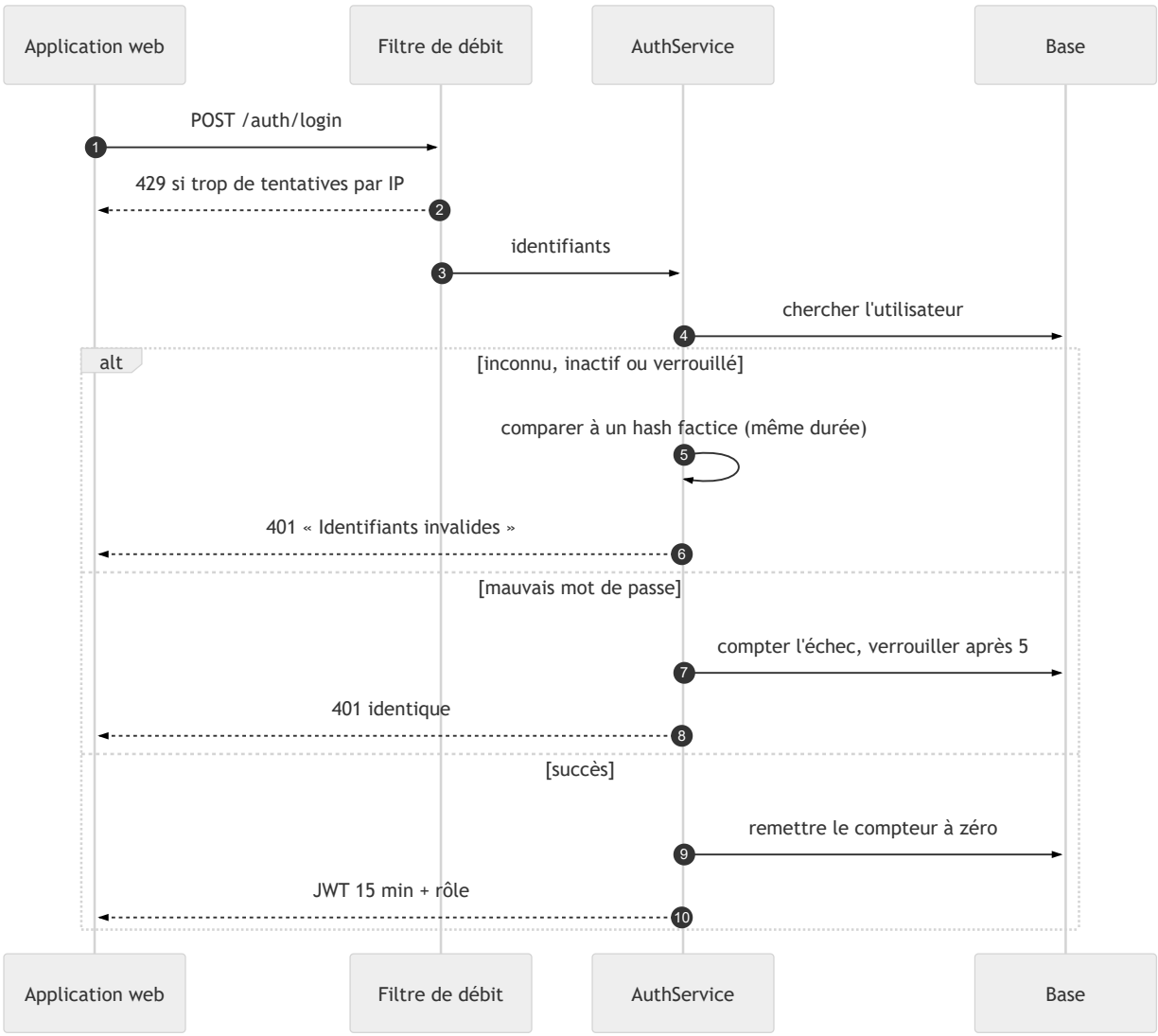


Pourquoi : environ 21 millions de lignes sur 30 jours pour 100 équipements × 5 métriques par minute (estimation à confirmer avec le CRI). **Comment** : partitionnement natif par mois, clé `(collected_at, id)` avec `id` entier généré, index BRIN sur l'horodatage. **Donc** la purge est instantanée et PostgreSQL seul suffit.

---

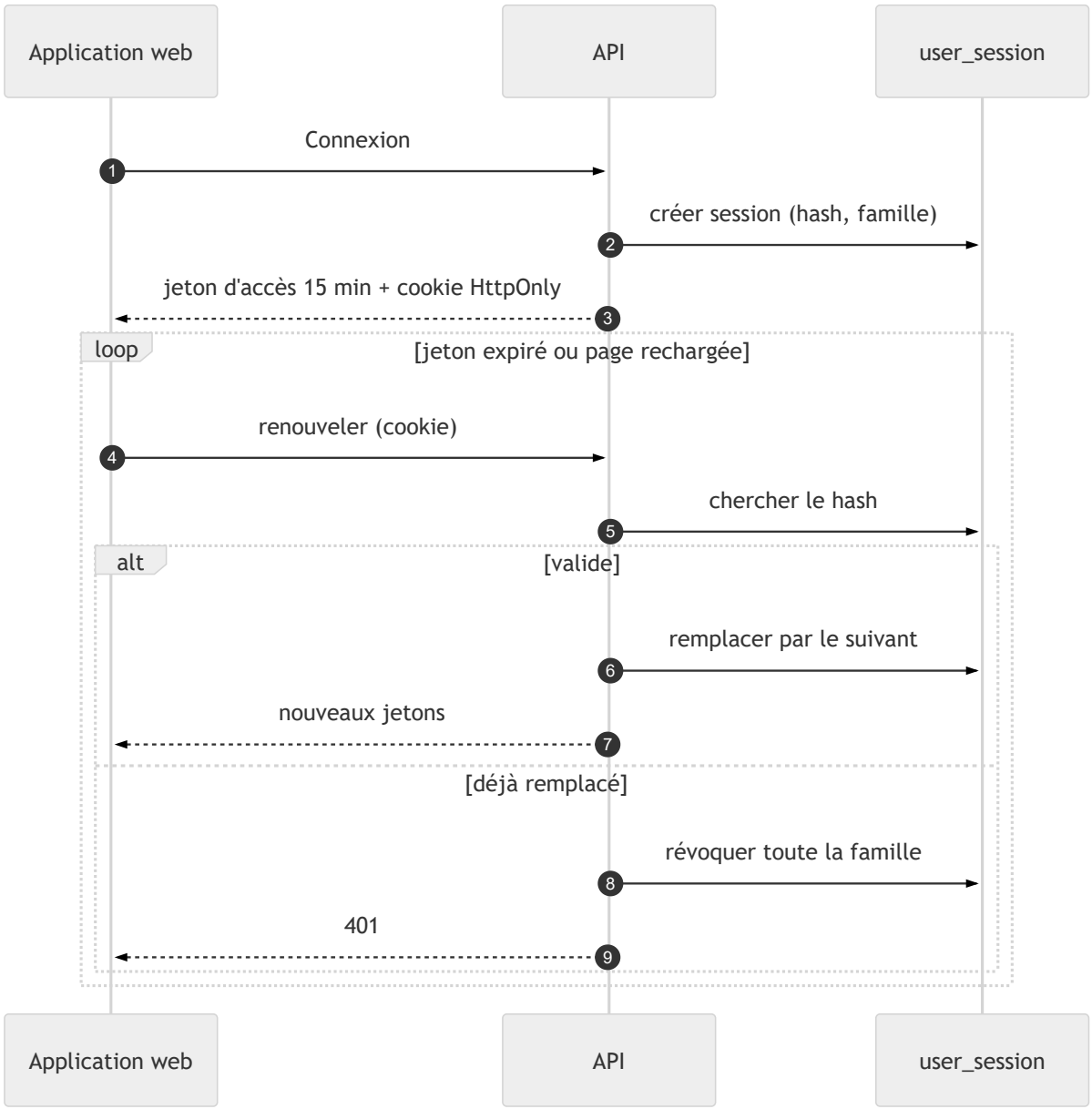
7. Comportements clés (séquences et états)

7.1 Connexion (réalisé)

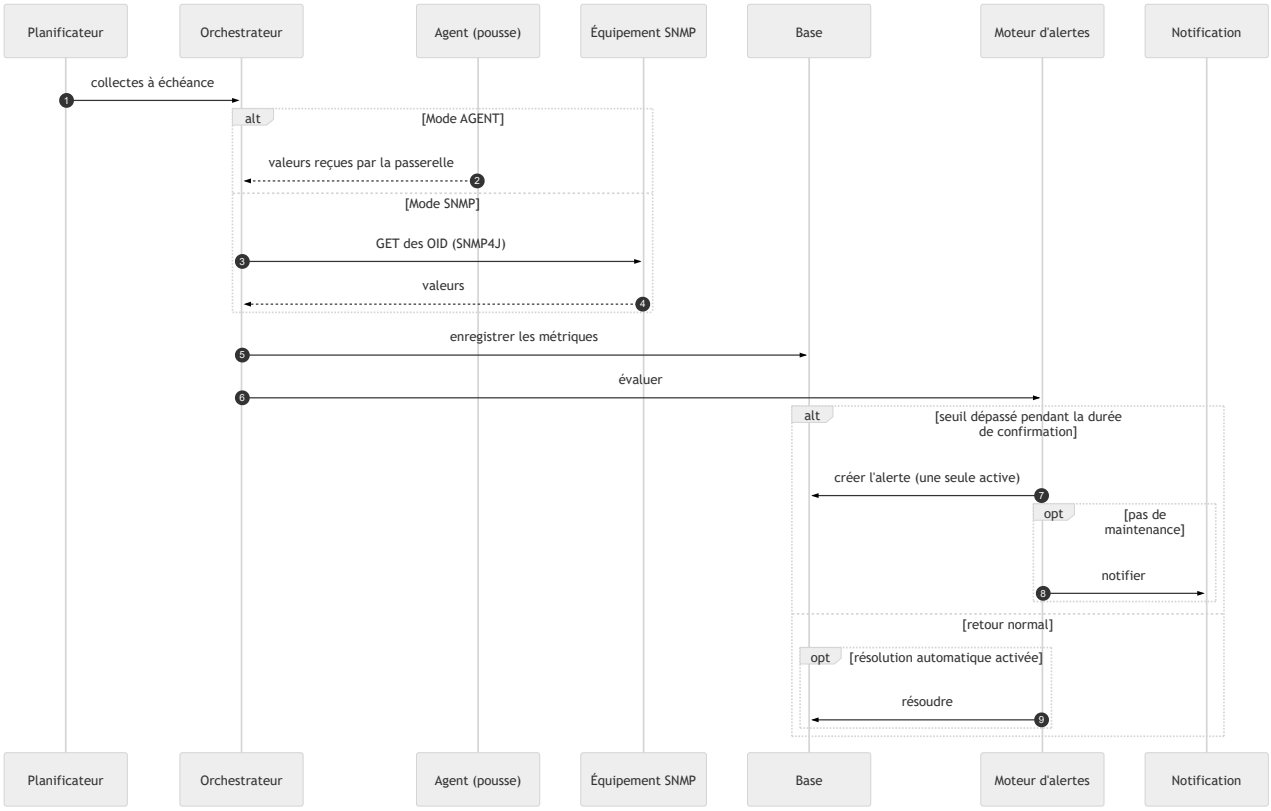


Pourquoi la réponse identique : un attaquant ne doit pas savoir si un compte existe. **Pourquoi le hash factice** : la durée de réponse ne trahit pas non plus l'existence du compte.

7.2 Renouvellement de jeton (réalisé, ADR-0005)

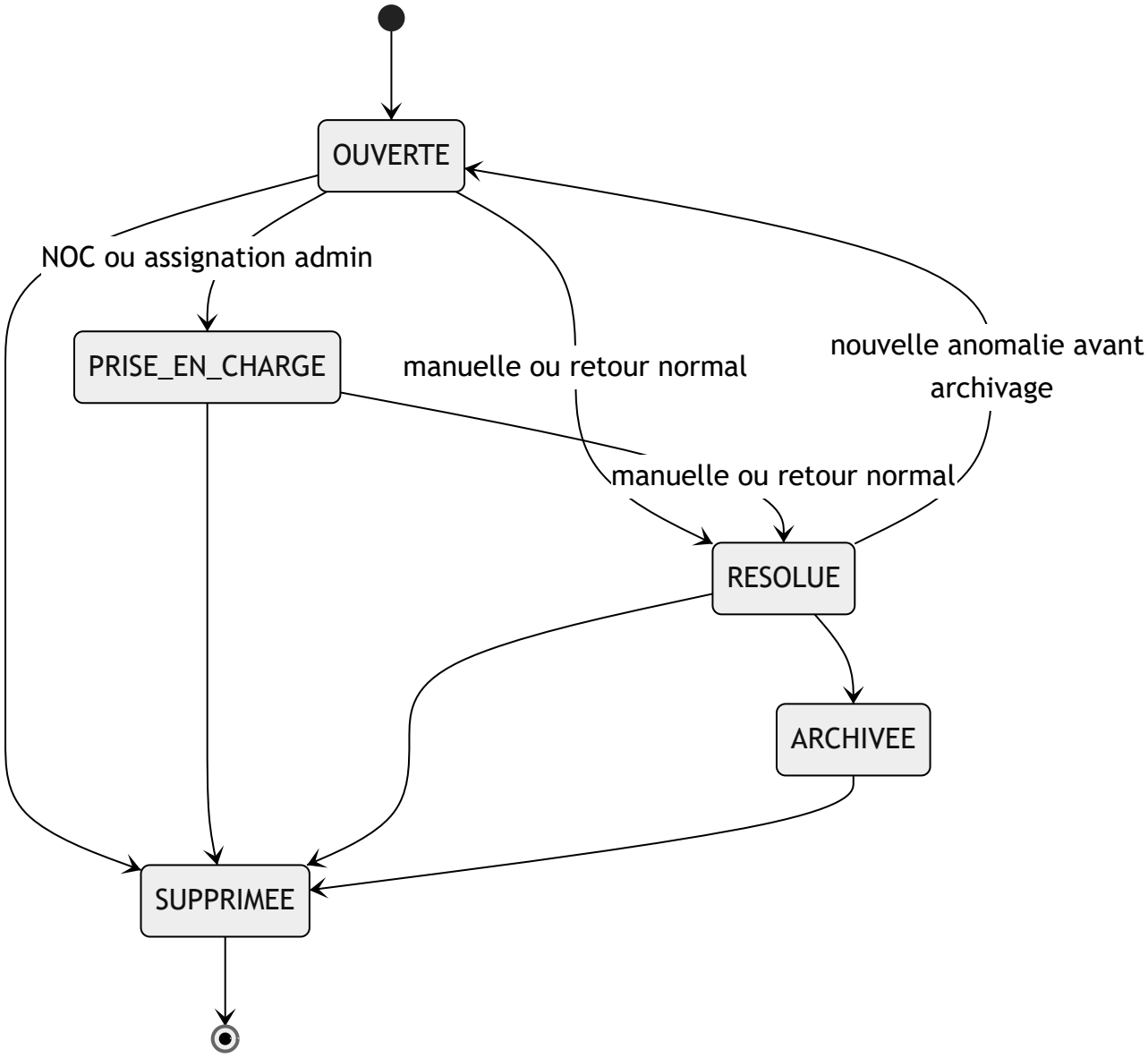


7.3 Collecte vers alerte (cible)



Pourquoi la durée de confirmation (`dureeConfirmation` dans `AlertRule`, déjà modélisée) : ignorer les pics brefs. Le seuil de retour à la normale distinct (hystérésis) n'existe pas encore (section 12).

7.4 Cycle de vie d'une alerte (modèle validé)



Sévérité indépendante de l'état : `INFO`, `WARNING`, `CRITICAL`. Le code de ce cycle n'existe pas encore (entités et énumération seulement).

---

8. Sécurité

Mesure	État	Pourquoi / comment
Mots de passe hachés, jamais journalisés	✅	`PasswordEncoder` Spring Security
JWT 15 min, API sans session (`STATELESS`)	✅	Pas d'état serveur à compromettre
CORS restreint aux origines configurées, en-têtes autorisés explicites	✅	`SecurityConfig`
En-têtes de sécurité (referrer, permissions…)	✅	`SecurityConfig`

Mesure	État	Pourquoi / comment
Limitation de débit et verrouillage progressif	✅	10 requêtes/60 s par IP, verrou 1 min doublé jusqu'à 30 min
Secrets de collecte par **référence** seulement	✅	`@JsonIgnore`, pas de `toString`, testé
Audit des actions sensibles	✅ partiel	inventaire audité ; rôles, alertes à venir
Analyse des secrets (gitleaks) et dépendances (Trivy, `npm audit`)	✅	CI
Jeton en mémoire + renouvellement par cookie avec rotation	✅	ADR-0005
SNMPv3 par défaut, v2c explicitement autorisé par équipement	🟡	ADR-0001

Corrigé : le jeton d'accès n'est plus dans `sessionStorage` mais en mémoire ; le renouvellement passe par un cookie `HttpOnly` avec rotation (une faille XSS ne vole plus de session durable). **CSRF** : désactivé car l'API n'utilise pas de cookie d'authentification pour l'instant ; l'ADR-0005 réintroduit un cookie pour le seul endpoint de renouvellement, avec `SameSite=Strict` et contrôle de l'origine.

---

9. Frontend

Pourquoi React 19 + Vite + Tailwind : écosystème dominant, démarrage rapide, composants testables ; choix de l'ADR-0001.

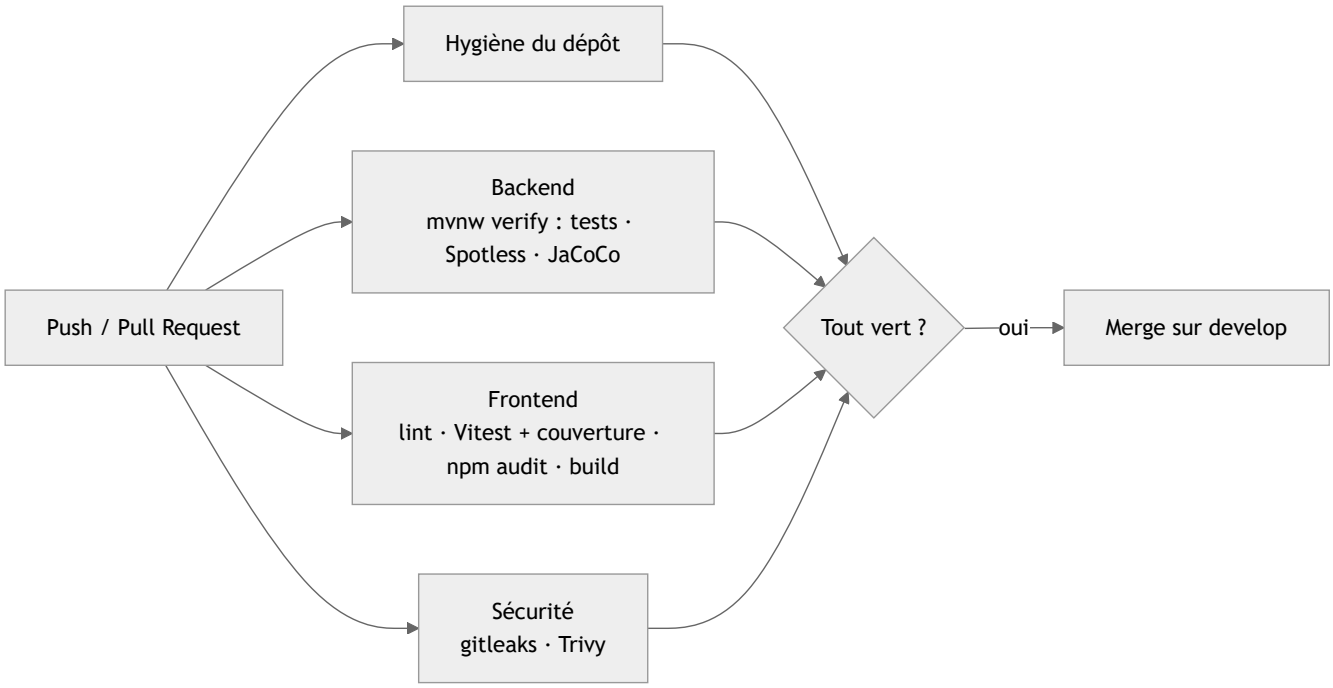
Comment : `App.jsx` déclare les routes ; `ProtectedRoute` redirige vers `/connexion` sans session ; `AuthContext` fournit l'utilisateur et le rôle ; `permissions.js` n'est qu'un confort d'affichage (le contrôle réel est côté serveur) ; `api/client.js` centralise l'URL de l'API et l'en-tête `Authorization`.

Route	Page	État
`/connexion`	Connexion	✅
`/tableau-de-bord`	Tableau de bord (mesure CPU synthétique)	✅ minimal
`/equipements`	Liste, création, modification, suppression	✅
Alertes, métriques, rapports, administration	—	🟣 maquettes dans `docs/design`

Les types TypeScript de l'API sont générés depuis `openapi.yaml`, pour éviter les écarts de contrat.

---

10. Qualité, CI/CD et exploitation



Contrôle	Seuil / outil	Pourquoi
Couverture backend	JaCoCo ≥ 80 % lignes, 60 % branches	Empêche l'érosion des tests
Couverture frontend	Vitest ≥ 75 % lignes	Idem
Format	Spotless (fichiers modifiés)	Diffs lisibles
Architecture	ArchUnit	Voir 5.2
Secrets	gitleaks (historique inclus)	Aucun secret dans Git
Dépendances	Trivy, `npm audit` (HIGH/CRITICAL)	Failles connues
Actions GitHub	épinglées par empreinte, Dependabot	Chaîne d'approvisionnement

Exploitation : journaux avec identifiant de corrélation, `/actuator/health` (sondes), documents `docs/devops/` (opérations, PostgreSQL, variables, workflow Git).

---

11. Décisions d'architecture et justifications

11.1 Index

ADR	Décision	Statut
0001	Stack : Spring Boot, React, PostgreSQL, Flyway, JWT	acceptée
0002	Construire sur mesure, en réutilisant des briques	proposée

ADR	Décision	Statut
0003	Tests sur un vrai PostgreSQL embarqué (sans Docker) au lieu de H2	acceptée, implémentée
0004	Métriques : partitionnement natif, clé entière, BRIN	acceptée, implémentée (hors purge)
0005	Jeton en mémoire + renouvellement par cookie avec rotation	acceptée, implémentée
0006	Java 21, SNMP4J, agent actif, SSE	proposée

11.2 Pourquoi / comment / donc, décision par décision

Construire plutôt qu'adopter (0002). *Pourquoi* : Zabbix, LibreNMS ou Prometheus + Grafana couvrent le besoin technique ; la question sera posée en soutenance. *Comment* : le projet est justifié par le cycle d'alerte, les rôles, l'audit et les rapports propres à l'EPT, avec briques réutilisées pour les protocoles et le stockage. *Donc* : périmètre assumé (pas de découverte automatique, pas de milliers d'équipements) et obligation de comparer avec ces outils dans le rapport. Les sources comparatives ne traitent pas le cas d'un projet pédagogique.

Tests sur PostgreSQL réel (0003). *Pourquoi* : H2 ne gère pas les index uniques partiels, donc les règles « une seule alerte active » ne sont jamais testées. Les sources convergent sur le risque de valider un code différent de celui de la production. *Comment* : PostgreSQL 16 embarqué (binaires lancés par la JVM, **sans Docker**, exigence du projet), mêmes migrations qu'en production, un test de violation par index. *Donc* : les tests tournent partout où tourne Java ; un seul dossier de migrations. Testcontainers a été écarté parce qu'il exige Docker.

Stockage des métriques (0004). *Pourquoi* : volume élevé, rétention différenciée. *Comment* : partitionnement natif mensuel. *Benchmarks* : les gains de TimescaleDB (insertions jusqu'à 20 fois plus rapides) viennent de l'éditeur et de PostgreSQL 14 ; une mesure isolée place le partitionnement natif avec BRIN à 15-30 % de TimescaleDB sauf pour les agrégats continus (source). Pour l'identifiant, `bigint` est le plus rapide, UUID v7 un compromis, UUID v4 le moins bon (ardentperf). *Donc* : natif + `bigint` ; TimescaleDB réévalué seulement après mesure sur le volume réel. Aucun benchmark indépendant relu n'existe.

Authentification (0005). *Pourquoi* : le jeton en `sessionStorage` est exposé au XSS. *Comment* : jeton d'accès court en mémoire, renouvellement par cookie `HttpOnly` avec rotation et détection de réutilisation, état dans `user_session`. *Donc* : une faille XSS ne vole plus de session durable ; le frontend et l'API doivent partager le même site. Sources : blogs ; relire l'OWASP et la documentation Spring Security 7 avant l'implémentation.

Exécution, collecte et temps réel (0006).

- *Java 21* : recommandé par Spring Boot 4, threads virtuels ; Java 25 est meilleur (épinglage corrigé à partir de Java 24) mais n'est pas encore vérifié avec les outils du projet. *Donc* : 21 maintenant, 25 ensuite. **Appliqué** : `pom.xml`, Dockerfile, CI, README.
- *SNMP4J* : SNMPv3, asynchrone, GETBULK, maintenu (3.12). Aucun benchmark comparatif trouvé ; derrière un port, donc remplaçable.
- *Agent actif* : les contrôles actifs de Zabbix réduisent la charge du serveur et l'immunisent contre les agents lents (témoignages, pas de mesure) ; les pare-feu n'autorisent que la sortie. Le backend ne scrute que le SNMP.
- *SSE* : flux serveur → navigateur suffisant, reconnexion automatique, aucune dépendance. WebSocket seulement si un besoin bidirectionnel apparaît.

11.3 Ce qui est bon et conservé

Monolithe modulaire vérifié par ArchUnit, ports pour la collecte, énumérations plutôt que sous-classes, contraintes en base, secrets par référence, limitation de débit, couverture imposée par la CI.

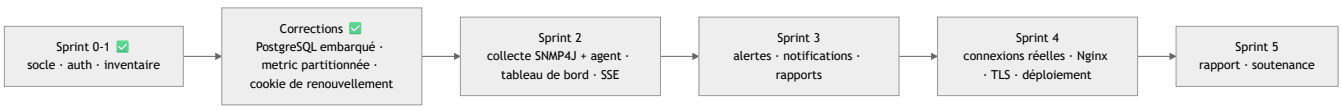
---

12. Écarts entre le code et la cible, risques

#	Écart ou risque	Gravité	Traitement
1	~~Tests sur H2 : index partiels non testés~~	Haute	**Corrigé** : ADR-0003, PostgreSQL embarqué ; 4 index partiels et partitionnement prouvés par des tests
2	~~`metric` non partitionnée, clé UUID~~	Haute	**Corrigé** : migration V9 (partitions mensuelles, clé `bigint`). Reste la purge des partitions expirées, à faire avec les agrégats
3	~~Jeton en `sessionStorage`~~	Moyenne	**Corrigé** : ADR-0005 implémenté. Limite connue : deux onglets qui renouvellent exactement en même temps peuvent déclencher la détection de réutilisation et fermer la session
4	~~Pas de seuil de retour à la normale (hystérésis)~~	Moyenne	**Corrigé** : `recovery_threshold` (V11) et méthodes `isBreachedBy` / `isRecoveredBy` testées
5	Pas de regroupement ni d'inhibition des alertes (un routeur en panne = 40 alertes)	Moyenne	à concevoir au Sprint 3
6	Collecteur « séparé » dans certains diagrammes, intégré dans l'ADR	Faible	décision : intégré au monolithe d'abord ; mise à jour des diagrammes de déploiement faite en partie
7	Anciens diagrammes et `00-overview.md` périmés conservés	Faible	marqués obsolètes dans le README des diagrammes
8	`CHANGELOG.md` en retard sur le travail réalisé	Faible	à mettre à jour
9	Prérequis externes : autorisation CRI pour tests réels, cibles SNMP, serveurs, SMTP, DNS/TLS	Haute (blocage)	à obtenir
10	Aucune date de sprint	Moyenne	à fixer avec l'encadrement

---

13. Feuille de route



Corrections faites : tests sur PostgreSQL embarqué, migration `metric` (V9), renouvellement de jeton (V10). Aussi faits : hystérésis des règles d'alerte (V11), agrégats horaires et purge des partitions expirées (V12), limitation de débit de `/auth/refresh` et nettoyage des sessions expirées. Restent : passage à Java 25,

regroupement des alertes, agrégats journaliers.

---

14. Annexes

14.1 Technologies

Couche	Choix	Version
Langage backend	Java	21
Framework	Spring Boot (MVC, Data JPA, Security, Validation, Actuator)	4.1.1
ORM	Hibernate	7.4
Base	PostgreSQL + Flyway	16
Limitation de débit	Bucket4j	8.21
API	springdoc OpenAPI	3.1.1
Tests backend	JUnit, Spring Boot Test, ArchUnit, PostgreSQL embarqué (zonky)	ArchUnit 1.5.1
Qualité backend	JaCoCo 0.8.15, Spotless 3.10	
Frontend	React 19, Vite 8, React Router 7, Axios 1, Tailwind 4	Node 24
Tests frontend	Vitest, Testing Library	
Conteneurs	Docker Compose, Mailpit	

14.2 Jeu canonique de diagrammes

Dans `docs/architecture/diagrams/` : cas d'utilisation (`01`), modèle persistant (`02`), séquences collecte, alertes et renouvellement de jeton (`03`), activité de collecte (`04`), cycle de l'alerte (`05`), composants cibles (`06`), déploiement cible (`07`), paquets (`10`). Les autres fichiers sont des versions antérieures conservées pour la traçabilité. Les fichiers `.mmd` s'affichent dans tout rendu Mermaid ; les images StarUML sont dans `docs/uml/` .

14.3 Documents associés

`docs/product-backlog.md` · `docs/architecture/persistence-mapping.md` · `docs/architecture/archunit.md` · `docs/devops/*` · `docs/api/openapi.yaml` · `docs/design/` (maquettes) · `deliverables/` (présentations).

14.4 Sources de la revue

- Stockage des séries temporelles : Timescale, Supabase, DEV Community
- Identifiants : ardentperf, PostgreSQL 18
- Tests : embedded-postgres, Testcontainers
- Authentification : OpenReplay, session et révocation
- Java et threads virtuels : InfoQ, BellSoft

- Collecte : SNMP4J, Zabbix actif/passif
- Temps réel : WebSocket vs SSE
- Modulithes : Frankel
- Outils existants : Zabbix vs LibreNMS

La plupart de ces sources sont des blogs ou des pages d'éditeurs, pas des études contrôlées : les chiffres sont des ordres de grandeur, à confirmer par des mesures sur la charge réelle du CRI.

Annexe A — Décisions d'architecture (ADR)

Texte intégral des six décisions d'architecture du projet, tel que présent dans `docs/adr/`.

ADR-0001 — Stack applicative Trans-DIC1

- Statut : acceptée
- Date : 2026-10-09
- Décision liée : #2

Contexte

Trans-DIC1 supervise les équipements réseau et serveurs de l'EPT. Le socle doit rester exploitable dans un réseau interne, éviter l'exposition de la topologie, supporter la collecte par agent Trans-DIC1 ou SNMP et fournir des builds reproductibles.

Décision

Domaine	Choix fixé
Backend	Java 21, Spring Boot 4.1.1, Maven Wrapper
Frontend	Node.js 24 LTS, npm 11.11.1, React 19, Vite 8
Interface	React Router 7, Axios 1, Tailwind CSS 4
Données	PostgreSQL 16 natif, migrations Flyway
Tests	JUnit/Spring Boot Test et H2 isolé ; Vitest/Testing Library
Exécution locale	Services natifs Java, Node.js et PostgreSQL, sans virtualisation

Le dépôt reste un monorepo : `backend/` porte l'API et les modules métier, `frontend/` l'interface, et les adaptateurs de collecte restent derrière des ports applicatifs afin de ne pas coupler le métier à l'agent Trans-DIC1 ou à SNMP. Chaque module backend sépare `controller`, `service`, `repository`, `domain` et `dto`.

Contrats

- Les endpoints métier sont versionnés sous `/api/v1`.
- Les erreurs utiliseront `application/problem+json` avec un code stable, un titre, un statut HTTP, une description et un identifiant de corrélation.
- Le client web centralise l'URL d'API et l'en-tête d'authentification.
- L'endpoint `/actuator/health` n'expose pas les détails internes au public.

Sécurité et rôles

L'authentification cible repose sur des jetons JWT courts (15 minutes) et un mécanisme de renouvellement révocable (8 heures maximum). Les rôles initiaux sont `ADMIN`, `NOC_OPERATOR` et `VIEWER`. Chaque connexion, mutation d'inventaire, acquittement/résolution d'alerte et changement de rôle doit produire une entrée d'audit horodatée et corrélée. Les communautés SNMP, tokens, mots de passe et adresses internes ne doivent jamais être placés dans Git ni dans les journaux.

Métriques et rétention

PostgreSQL constitue le stockage initial. Les métriques brutes sont conservées 30 jours, les agrégats horaires 13 mois et les agrégats journaliers 3 ans. Le partitionnement temporel et la purge automatisée seront introduits avec le module de collecte. Cette stratégie sera réévaluée à partir du volume observé avant toute adoption d'une extension spécialisée.

Contraintes d'exploitation

- Les collectes SNMP privilégient SNMPv3 ; SNMPv2c reste une compatibilité explicitement autorisée par équipement.
- Les appels à l'agent Trans-DIC1 sont effectués côté serveur avec authentification, délais, reprises bornées et limitation de débit.
- Les collecteurs doivent fonctionner sans accès Internet et ne publier aucune donnée de topologie.
- Les images et dépendances doivent pouvoir être mises en cache dans l'infrastructure EPT.
- Le développement local ne dépend pas de Docker. Le mode de déploiement cible sera fixé après validation de l'hébergement EPT.

Conséquences

Les versions et frontières sont désormais explicites. Cette ADR constitue la décision de référence ; toute modification structurante devra être portée par une nouvelle ADR.

Précisions ultérieures

- ADR-0002 : justification du développement sur mesure.
- ADR-0003 : les tests de persistance passent de H2 à un vrai PostgreSQL embarqué, sans Docker (remplace la ligne « Tests » ci-dessus pour la base de données).
- ADR-0004 : partitionnement natif des métriques, à décider avant le module de collecte.
- ADR-0005 : jeton d'accès en mémoire et renouvellement par cookie avec rotation.

ADR-0002 — Construire Trans-DIC1 plutôt qu'adopter une solution de supervision existante

- Statut : proposée
- Date : 2026-10-28

Contexte

Des outils open source couvrent déjà l'essentiel du besoin : Zabbix (collecte, alertes, tableaux de bord, passage à l'échelle par proxys), LibreNMS (découverte SNMP automatique, mise en route rapide) ou la combinaison Prometheus + `snmp_exporter` + Grafana pour la visualisation. Les comparatifs consultés ne justifient pas un développement sur mesure pour le seul besoin technique de supervision. Ils ne traitent pas non plus le cas d'un projet de fin d'études.

Cette question sera posée en soutenance et lors de la revue par le CRI. Elle doit avoir une réponse écrite.

Options

Option	Avantages	Limites
A. Adopter Zabbix ou LibreNMS	Fonctionnel dès le premier jour, communauté, modèles d'équipements	Pas de valeur de conception pour le projet, personnalisation limitée du cycle de vie des alertes et des rôles EPT
B. Prometheus + Grafana	Visualisation très riche, stockage séries temporelles éprouvé	Assemblage de plusieurs outils, pas de gestion d'inventaire ni de cycle d'incident prise en charge/résolution
C. Construire Trans-DIC1	Cycle d'alerte, rôles, audit et rapports adaptés au CRI, objet d'étude du projet	Coût de développement et de maintenance, couverture protocolaire plus étroite
D. Hybride : construire le métier, réutiliser des briques	Réutiliser une bibliothèque SNMP et PostgreSQL plutôt que réécrire les protocoles	Dépendances à choisir et à maintenir

Décision

Retenir **l'option D**. Trans-DIC1 reste une application sur mesure, car son objet est l'inventaire, le cycle de vie des alertes (`OUVERTE` → `PRISE_EN_CHARGE` → `RESOLUE` → `ARCHIVEE`), les rôles, l'audit et les rapports propres à l'EPT. Les protocoles et le stockage ne sont pas réécrits : bibliothèque SNMP existante, PostgreSQL, Spring Security.

Périmètre explicitement exclu

Pas de découverte automatique du réseau, pas de bibliothèque de modèles par constructeur, pas de supervision de milliers d'équipements. Au-delà de ce périmètre, Zabbix ou LibreNMS redeviennent le bon choix.

Conséquences

- La justification est écrite pour la soutenance.
- Le rapport doit comparer Trans-DIC1 à Zabbix et LibreNMS sur des critères mesurables : fonctions couvertes, effort de mise en route, limites.
- À vérifier avec le CRI : Zabbix ou LibreNMS sont-ils déjà en place ? Si oui, Trans-DIC1 doit pouvoir cohabiter, par exemple en important des mesures.
- Sources : Zabbix vs LibreNMS, Zabbix vs Grafana, collecteurs SNMP auto-hébergés.

ADR-0003 — Tests d'intégration sur un vrai PostgreSQL embarqué, sans Docker

- Statut : acceptée et implémentée (28 octobre 2026)
- Date : 2026-10-28
- Complète : ADR-0001 (« Tests : JUnit/Spring Boot Test et H2 isolé »)

Contexte

Les tests du backend utilisaient H2 en mode PostgreSQL. H2 ne gère pas les index uniques partiels : le dossier `db/postgres` (`V8__partial_unique_indexes.sql`) n'était donc jamais chargé par les tests. Les règles les plus importantes du modèle reposent pourtant sur ces index :

- une seule alerte active par équipement et règle (`uq_alert_one_active_*`) ;
- un seul agent actif par équipement ;
- une seule affectation active par alerte.

La table des métriques partitionnée (ADR-0004) ne peut pas non plus exister sous H2.

Une première version de cette décision utilisait Testcontainers. Elle est écartée : **les tests ne doivent pas dépendre de Docker** (poste de développement, environnement EPT sans virtualisation, cohérent avec l'ADR-0001 qui exige un développement local sans Docker).

Décision

1. Les tests s'exécutent sur un **vrai PostgreSQL 16 embarqué** : la bibliothèque `io.zonky.test:embedded-postgres` fournit les binaires natifs de PostgreSQL et les lance comme un simple processus de la JVM. Aucun Docker, aucune installation, aucun service à démarrer. Les binaires sont figés sur la version 16.15 par le BOM `embedded-postgres-binaries-bom`, comme la production (PostgreSQL 16).
2. Un seul serveur est démarré pour toute l'exécution des tests (`EmbeddedPostgresInitializer` , déclaré dans `META-INF/spring.factories`) et relié à tous les `@SpringBootTest` sans les modifier. Il s'arrête avec la JVM.
3. Flyway applique **toutes** les migrations, depuis un **unique dossier** `db/migration` : V8 (index partiels) y a été déplacé sans modification de contenu, donc sans changer son empreinte Flyway.
4. H2 est supprimé des dépendances.
5. `PartialUniqueIndexTests` prouve chaque index partiel par une violation réelle ; `MetricPartitioningTests` prouve le partitionnement ; `PersistenceModelTests` vérifie les violations de contraintes par code SQLSTATE de classe 23 (indépendant du moteur).
6. La CI exécute les tests sur les runners Ubuntu sans configuration supplémentaire.

Conséquences

- Premier lancement : téléchargement des binaires PostgreSQL (environ 10 Mo) dans le dépôt Maven local, puis lecture en cache.
- PostgreSQL ne peut pas être lancé en tant qu'administrateur sous Linux : les runners GitHub et les postes de développement l'exécutent sous un compte normal, ce qui convient.

- Les tests sont plus lents de quelques secondes au démarrage (initialisation du cluster).
- Sources : embedded-postgres (zonky), Testcontainers — remplacer H2 pour le risque de valider un code différent de la production. Les sources ne traitent pas précisément les index partiels : les tests de l'étape 5 en sont la preuve.

ADR-0004 — Stockage des métriques : partitionnement natif PostgreSQL

- Statut : acceptée et implémentée (migrations V9 et V12)
- Date : 2026-10-28
- Précise : ADR-0001, section « Métriques et rétention »

Contexte

L'ADR-0001 fixe la rétention (30 jours bruts, 13 mois d'agrégats horaires, 3 ans d'agrégats journaliers) et renvoie le partitionnement au module de collecte. Or la table `metric` existe déjà (V7) sans partitionnement, et son identifiant est un UUID.

Ordre de grandeur (estimation, à remplacer par les valeurs réelles du CRI) : 100 équipements × 5 métriques × 1 mesure par minute ≈ 720 000 lignes par jour, soit environ 21 millions de lignes sur 30 jours. PostgreSQL le supporte à condition que la table soit partitionnée, car la purge devient un `DROP` de partition au lieu d'un `DELETE` massif.

Une table partitionnée impose que la clé de partitionnement figure dans la clé primaire. Changer cela après avoir rempli la table est coûteux. La décision doit donc précéder le module de collecte.

Éléments de benchmark

- Insertion et requêtes : les chiffres favorables à TimescaleDB (insertions jusqu'à 20 fois plus rapides, billet Timescale) viennent de l'éditeur et portent sur PostgreSQL 14. Une mesure indépendante plus récente et non relue (DEV Community) place le partitionnement natif avec BRIN à 15-30 % de TimescaleDB sur la plupart des requêtes ; l'écart majeur vient des agrégats continus.
- Identifiant : `bigint` insère le plus vite, UUID v7 est un compromis, UUID v4 gonfle l'index de 40 à 100 % selon une source et dégrade les insertions quand l'index dépasse la mémoire cache (ardentperf, comparatif PostgreSQL 18). Les pourcentages dépendent de la charge ; le sens du classement est cohérent entre sources.
- Pour ce projet (une vingtaine de millions de lignes sur 30 jours, serveur EPT, peu de personnes pour l'exploiter), l'écart de performance ne justifie pas une extension : les agrégats sont des tables alimentées par une tâche planifiée.

Options

Option	Avantages	Limites
A. Table ordinaire + `DELETE` périodique	Aucun changement	Purge lente, fragmentation, requêtes de plus en plus lentes
B. Partitionnement déclaratif natif + `pg_partman`	Pur PostgreSQL, portable, purge par `DROP`, pas de dépendance exotique	Agrégats à écrire en SQL, automatisation à configurer
C. TimescaleDB	Hypertables, compression, `time_bucket`, agrégats continus	Extension à installer sur le serveur EPT, outillage moins portable

Les sources comparatives sont des pages d'éditeurs et des mesures isolées ; aucun benchmark indépendant récent et relu n'a été trouvé. Le choix se fait donc sur la contrainte d'exploitation (serveur EPT à maîtrise limitée) et sur l'écart modeste constaté hors agrégats continus, pas sur la performance brute. Les performances seront mesurées sur le volume réel avant toute adoption de TimescaleDB.

Décision

1. Retenir **l'option B** : partitionnement par intervalle sur l'horodatage de collecte, une partition par mois, créée et supprimée par `pg_partman` (ou, à défaut, par une tâche planifiée interne).
2. Clé primaire de `metric` : `(collected_at, id)` avec `id` `bigint` **généré** (`GENERATED ALWAYS AS IDENTITY`). `metric` est une table de faits interne, jamais exposée par identifiant et sans clé étrangère entrante ; l'identifiant opaque d'un UUID n'apporte rien ici. Les autres entités gardent leurs UUID.
3. Index : `(equipment_id, metric_key, collected_at DESC)` par partition pour les lectures par équipement, et un index **BRIN** sur `collected_at` pour les balayages temporels (rapports, agrégation), très compact sur des données insérées dans l'ordre.
4. Les agrégats horaires et journaliers sont des tables distinctes alimentées par une tâche planifiée, avec leur propre rétention.
5. Le choix de TimescaleDB est réévalué seulement si les agrégats deviennent trop lents sur le volume réel. Aucune dépendance à `time_bucket` n'est introduite d'ici là.
6. La migration qui convertit `metric` est écrite **avant** la première insertion réelle. Les données de démonstration de V4 sont recréées, pas converties.

Conséquences

- Fait : migration V9 (clé primaire `(collected_at, id)` , séquence `metric_id_seq` , partition par défaut, fonction `ensure_metric_partition`) et entité `Metric` à identifiant `Long` . `MetricPartitionService` crée chaque jour les partitions des 3 mois suivants (`pg_partman` non retenu : aucune extension à installer).
- `MetricResponse.id` passe de UUID à entier : contrat OpenAPI et types TypeScript mis à jour.
- Fait aussi (V12) : table `metric_hourly` alimentée chaque heure par `MetricRetentionService` (recalcul des deux dernières heures pour les mesures en retard), puis suppression quotidienne des partitions mensuelles dont la borne haute a plus de 30 jours **et** est couverte par les agrégats, et des agrégats de plus de 13 mois. Une mesure n'est donc jamais supprimée avant d'être agrégée ; avec des partitions mensuelles, les mesures brutes sont conservées entre 30 et environ 60 jours.
- Reste à faire : agrégats journaliers (3 ans).
- La purge de la rétention devient une opération de partition, testée sur PostgreSQL réel (voir ADR-0003).
- Le diagramme de classes persistant et l'ERD indiquent la nouvelle clé.
- Sources : pg_partman vs hypertables, guide Supabase de migration vers pg_partman, fil pgsql-general.

ADR-0005 — Authentification : jeton d'accès court et renouvellement par cookie avec rotation

- Statut : acceptée et implémentée (migration V10, `AuthController`, `AuthService`, frontend)
- Date : 2026-10-28
- Précise : ADR-0001, section « Sécurité et rôles »

Contexte

L'ADR-0001 prévoit un JWT de 15 minutes et un renouvellement révocable de 8 heures au plus. L'état actuel du code :

- le jeton est conservé par le frontend dans `sessionStorage` (`frontend/src/auth/session.js`), donc lisible par tout script exécuté dans la page en cas de faille XSS ;
- la table `user_session` existe (hash du jeton, adresse IP, expiration, révocation) mais aucun flux de renouvellement ne l'utilise encore.

Les sources consultées convergent : jeton d'accès de courte durée gardé en mémoire, jeton de renouvellement dans un cookie `HttpOnly`, `Secure`, `SameSite`, rotation à chaque usage, détection de réutilisation, état de révocation côté serveur. Ce sont des blogs et des guides, pas la norme : les cheat sheets OWASP et la documentation Spring Security 7 doivent être relues avant l'implémentation.

Décision

1. **Jeton d'accès** : JWT de 15 minutes, renvoyé dans le corps de la réponse, conservé **en mémoire** par le frontend (jamais dans `localStorage` ni `sessionStorage`).
2. **Jeton de renouvellement** : valeur aléatoire opaque de 256 bits, envoyée dans un cookie `HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth`. Seul son hash est stocké dans `user_session`.
3. **Rotation** : chaque renouvellement invalide le jeton présenté et en émet un nouveau, dans la limite de 8 heures depuis la connexion.
4. **Détection de réutilisation** : présenter un jeton déjà remplacé révoque toute la chaîne de la session et produit une entrée d'audit.
5. **Révocation** : déconnexion, changement de rôle, désactivation du compte ou réinitialisation du mot de passe révoquent les sessions de l'utilisateur.
6. **CSRF** : `SameSite=Strict` plus vérification de l'en-tête `Origin` sur le seul endpoint de renouvellement ; les autres endpoints exigent l'en-tête `Authorization`, donc ne sont pas exposés au CSRF par cookie.
7. **Rechargement de page** : le frontend appelle l'endpoint de renouvellement au démarrage pour récupérer un jeton d'accès, ce qui remplace la persistance en `sessionStorage`.
8. La limitation de débit et le verrouillage progressif de SEC-01 restent inchangés.

Conséquences

- Une faille XSS ne permet plus de voler une session durable.
- Le déploiement doit servir le frontend et l'API sous le même site (Nginx en proxy) pour que `SameSite=Strict` fonctionne.

- Il faut modifier `user_session` : colonne de chaîne (`family_id`), `replaced_by` , et un index sur le hash.
- Le flux est décrit dans token-refresh.mmd.
- Mise en œuvre : endpoints `POST /api/v1/auth/refresh` et `/logout` ; CORS avec identifiants pour les seules origines configurées ; côté navigateur, un renouvellement à la fois (`refresh.js`) pour ne pas déclencher la détection de réutilisation, relance unique d'une requête après un 401, renouvellement 30 secondes avant l'expiration.
- Limite connue : deux onglets qui renouvellent exactement au même instant peuvent faire détecter une réutilisation et fermer la session. Une courte période de grâce pourra être ajoutée si cela gêne le NOC.
- Fait aussi : limitation de débit de `/refresh` (trois fois le seuil de connexion, par adresse) et suppression quotidienne des sessions expirées depuis plus de 7 jours (`SessionCleanupService`).
- Non fait : nombre maximal de sessions par utilisateur.
- Sources : OpenReplay — cookies vs localStorage, checklist de stockage des jetons, gestion de session et révocation.

ADR-0006 — Exécution, collecte et temps réel : Java 21, SNMP4J, agent actif, SSE

- Statut : proposée
- Date : 2026-10-28
- Précise : ADR-0001 (backend, collecte) ; remplace WebSocket du backlog

Contexte

Quatre choix techniques restaient implicites ou contradictoires entre le backlog, les diagrammes et le code. Les benchmarks publics sont rares sur ces sujets ; chaque décision indique ce qui est établi et ce qui ne l'est pas.

Décisions

1. Java 21 (et non 17 ni 25)

- Spring Boot 4 supporte Java 17 à 25/26 ; le 21 est le niveau recommandé et active les threads virtuels (guide).
- Les threads virtuels de Java 21 peuvent rester « épinglés » dans un bloc `synchronized`, ce qui a été corrigé à partir de Java 24 (InfoQ). Java 25 est donc techniquement meilleur pour de nombreux appels bloquants simultanés.
- Retenu : **Java 21 maintenant**, car il est installé et vérifié sur le poste et la CI, et que la charge réelle (quelques centaines d'équipements) se traite avec un pool borné d'exécuteurs. **Passage à Java 25 prévu** dès que la CI, JaCoCo, Spotless et ArchUnit sont validés dessus ; il ne demande qu'un changement de version.
- Les threads virtuels ne sont pas activés par défaut : à activer seulement après mesure sur la collecte.

2. Bibliothèque SNMP : SNMP4J 3.x

- SNMP4J gère SNMPv3 (authentification SHA, chiffrement AES), les requêtes synchrones et asynchrones, GETBULK et le multi-threading ; la version courante sur Maven Central est la 3.12 (Maven Central).
- Aucun benchmark indépendant de bibliothèques SNMP Java n'a été trouvé. Le choix repose sur les fonctions (SNMPv3 exigé par l'ADR-0001), la maintenance et le fait que l'alternative commerciale (iReasoning) est propriétaire.
- L'accès reste derrière un port applicatif : SNMP4J est remplaçable sans toucher au métier.

3. Agent Trans-DIC1 en mode actif (l'agent envoie)

- Pour les serveurs porteurs d'un agent, l'agent **envoie** ses mesures à l'API (authentification par jeton d'agent, TLS). Le backend n'ouvre pas de connexion vers chaque serveur.
- Raisons : d'après les échanges des utilisateurs et du créateur de Zabbix, les contrôles actifs réduisent la charge du serveur et le rendent insensible aux lenteurs d'un agent ; un agent lent bloque sinon un processus d'interrogation (blog Zabbix, forum). Ce sont des témoignages et non des mesures, sur des

versions anciennes. Le même article note que l'écart est faible pour peu de mesures rares.
- Aussi : les pare-feu des sites n'ont à autoriser que la sortie vers l'API, ce qui convient à l'ADR-0001 (pas d'exposition de la topologie).
- Le backend **scrute** uniquement les équipements SNMP, qui ne peuvent pas pousser.
- Conséquence sur les diagrammes : `collection-and-alert.mmd` ne montre plus l'orchestrateur appelant l'agent ; la passerelle des agents reçoit les mesures.
- Mis en œuvre : un administrateur enregistre l'agent (`POST /api/v1/equipements/{id}/agent`) et reçoit un jeton affiché une seule fois (seule son empreinte SHA-256 est stockée) ; l'agent envoie ses lots à `POST /api/v1/agent/measures` avec ce jeton. Une chaîne de sécurité dédiée (`/api/v1/agent/**`) n'accepte que les jetons d'agent : un JWT d'utilisateur n'ouvre pas cette API, et un jeton d'agent n'ouvre pas l'API des utilisateurs. Enregistrer un nouvel agent révoque le précédent.

4. Temps réel : SSE (Server-Sent Events), pas WebSocket

- Le tableau de bord n'a besoin que de flux serveur vers navigateur. SSE utilise HTTP standard, se reconnecte automatiquement et s'implémente avec `SseEmitter` sans nouvelle dépendance (comparatif, OneUptime).
- WebSocket n'est introduit que si un besoin bidirectionnel apparaît. Les sources sont des blogs, sans mesure de charge pour ce cas.
- Le flux est authentifié par le jeton d'accès ; comme `EventSource` ne permet pas d'en-tête personnalisé, le frontend utilise `fetch` avec lecture du flux.

Conséquences

- `pom.xml` , Dockerfile, CI et README passent à Java 21 (appliqué).
- Le backlog et les diagrammes de composants et C4 remplacent WebSocket par SSE (appliqué).
- Le module de collecte (Sprint 2) ajoute SNMP4J et un endpoint de réception des mesures d'agent.

Annexe B — Catalogue des diagrammes

Catalogue des diagrammes

Ce catalogue complète `../00-overview.md`. Les éléments qualifiés « cible » proviennent de l'ADR et du backlog ; ils ne sont pas encore implémentés au Sprint 0.

Jeu canonique validé après entretien métier

Les fichiers ci-dessous constituent la version à utiliser pour la suite de la conception. Les autres fichiers du catalogue conservent les propositions antérieures à titre de traçabilité.

- `01-use-cases/trans-dic1-use-cases.mmd` : trois rôles humains (Lecteur, NOC, Administrateur, par généralisation) ; agent Trans-DIC1 et service e-mail comme systèmes externes ; aucun équipement n'est modélisé comme acteur.
- `02-domain/persistence-model-uml.mmd` : modèle UML des seules classes persistées, sans clés étrangères dupliquées dans les attributs.
- `03-sequences/collection-and-alert.mmd` : collecte configurable par agent ou SNMP, puis évaluation des alertes.
- `03-sequences/alert-handling.mmd` : prise en charge par un NOC, assignation par un Administrateur et résolution.
- `03-sequences/token-refresh.mmd` : connexion, renouvellement avec rotation, détection de réutilisation et déconnexion (ADR-0005).
- `04-activities/metric-collection.mmd` : cycle planifié avec intervalle configurable, reprises, maintenance et résolution automatique.
- `05-states/alert-lifecycle.mmd` : états `OUVERTE`, `PRISE_EN_CHARGE`, `RESOLUE`, `ARCHIVEE` et `SUPPRIMEE`.
- `06-components/target-components.mmd` : composants cibles sans dépendance à Zabbix.
- `07-deployment/trans-dic1-target.mmd` : déploiement cible de la plateforme, de l'agent et des équipements SNMP.
- `10-packages/packages.mmd` : dépendances entre les paquets fonctionnels et techniques.

Le relevé de validation de l'issue #5 est dans `VALIDATION-S0.md`.

Les diagrammes de classes, de séquence et d'états utilisent les syntaxes UML natives de Mermaid. Les cas d'utilisation, activités, composants, déploiement et paquets utilisent `flowchart`, Mermaid ne proposant pas de syntaxe UML native pour ces vues.

Métadonnées par diagramme

`01-use-cases/global-use-cases.mmd`

> Proposition initiale remplacée, après entretien métier, par `validated-global-use-cases.mmd`, `monitoring-use-cases.mmd` et `administration-use-cases.mmd`. La synthèse validée se trouve dans `../../USE-CASES-VALIDES.md`.

- **Nom :** cas d'utilisation global.
- **Objectif :** relier rôles externes et objectifs métier dans la frontière Trans-DIC1.
- **Question :** qui utilise le système, pour obtenir quel résultat ?
- **Périmètre :** parcours décrits par l'ADR et les US05 à US22.
- **Exclus :** clics UI, endpoints et détails techniques.
- **Sources dépôt :** ADR-0001, backlog, `App.jsx`.
- **Règles appliquées :** acteurs externes, cas nommés par objectifs ; aucun `include` / `extend` non prouvé.
- **Hypothèse :** le planificateur est traité comme acteur logique.
- **Support :** Mermaid n'offre pas de use-case natif ; `flowchart` est une substitution visuelle documentée.

`02-domain/domain-overview.mmd`

- **Nom :** modèle conceptuel métier cible.
- **Objectif :** présenter entités, valeurs, comportements et relations essentiels sans ORM ni framework.
- **Question :** quels objets fondamentaux collaborent pour superviser l'infrastructure ?
- **Périmètre :** inventaire, monitoring, alerting, notification, reporting, identité et audit.
- **Exclus :** DTO, contrôleurs, JPA, HTTP, timestamps techniques génériques.
- **Sources dépôt :** modèle initial, ADR-0001, US02, US13 à US19.
- **Règles appliquées :** multiplicités aux deux extrémités ; composition limitée au service dépendant de l'équipement ; opérations métier plutôt que setters.
- **Hypothèses :** un équipement appartient à un site ; un service supervisé n'existe pas sans équipement.

`02-domain/design-model.mmd`

- **Nom :** modèle de conception cible.
- **Objectif :** séparer boundary, application, domaine, ports et adaptateurs.
- **Question :** comment implémenter les scénarios sans coupler le domaine à Spring, JPA, SNMP ou Zabbix ?
- **Périmètre :** collecte et alertes, les flux les plus risqués.
- **Exclus :** détails React, classes Spring générées et méthodes CRUD exhaustives.
- **Sources dépôt :** ADR-0001 (ports applicatifs), US03, US07, US09, US13, US14, US16, US21, US22.
- **Règles appliquées :** réalisation pour les implémentations de ports, dépendances pour les usages, aucune classe d'infrastructure dans le modèle métier.
- **Hypothèse :** le collecteur pourra être un processus séparé ; ce choix reste ouvert.

`03-sequences/collect-measure-and-open-alert.mmd`

- **Nom :** séquence collecte vers alerte.
- **Objectif :** montrer l'ordre des interactions du scénario central.
- **Question :** comment une observation devient-elle métrique puis alerte notifiée ?
- **Périmètre :** chemin nominal, déduplication et absence de dépassement.
- **Exclus :** sérialisation, requêtes SQL et détails des protocoles.
- **Sources dépôt :** modèle initial, US09, US13, US16, US21, US22.
- **Règles appliquées :** temps vertical ; `loop` pour les métriques ; `alt` pour les résultats exclusifs.
- **Hypothèse :** une clé d'occurrence permet la déduplication.

`03-sequences/acknowledge-and-resolve-alert.mmd`

- **Nom :** séquence de traitement d'une alerte.
- **Objectif :** localiser autorisation, invariant de transition, persistance et audit.
- **Question :** comment l'opérateur acquitte-t-il puis résout-il une alerte ?
- **Périmètre :** rôle NOC et API cible.
- **Exclus :** renouvellement JWT et implémentation UI.
- **Sources dépôt :** ADR-0001, US05, US14, US15.
- **Règles appliquées :** messages orientés intention ; alternatives autorisé/interdit et valide/invalide.
- **Hypothèse :** résolution directe d'une alerte ouverte autorisée, à valider métier.

`04-activities/incident-lifecycle.mmd`

- **Nom :** activité de gestion d'un incident.
- **Objectif :** rendre visibles décisions et reprises du processus complet.
- **Question :** quelles décisions mènent d'une collecte à une résolution ?
- **Périmètre :** collecte, validation, seuil, déduplication, notification et clôture.
- **Exclus :** composants responsables, couverts par les séquences.
- **Sources dépôt :** US09, US13, US14, US16, règles d'exploitation.
- **Règles appliquées :** actions et décisions distinctes, début et fin explicites.
- **Hypothèse :** le retour à la normale est un prérequis de résolution opérationnelle.

`05-states/alert-state.mmd`

- **Nom :** machine à états de l'alerte.
- **Objectif :** interdire les transitions incohérentes.
- **Question :** dans quels états une alerte peut-elle se trouver et quels événements la font évoluer ?
- **Périmètre :** cycle métier documenté.
- **Exclus :** échec de notification et disponibilité d'équipement.
- **Sources dépôt :** modèle initial, ADR-0001, US14.
- **Règles appliquées :** états nommés par situations, transitions par événements, état terminal explicite.
- **Hypothèse :** résolution directe depuis OUVERTE ; pas de réouverture.

`06-components/components.mmd`

- **Nom :** composants logiciels cibles.
- **Objectif :** montrer les blocs et protocoles majeurs.

- **Question :** quelles responsabilités macroscopiques composent Trans-DIC1 ?
- **Périmètre :** frontend, backend logique, persistance et systèmes externes.
- **Exclus :** classes et détails de déploiement.
- **Sources dépôt :** README, ADR-0001, architecture DevOps, backlog.
- **Règles appliquées :** composants à granularité cohérente, interfaces annotées par protocole.
- **Hypothèse :** séparation logique des composants dans un backend initialement monolithique.

`07-deployment/deployment-current.mmd`

- **Nom :** déploiement local actuel.
- **Objectif :** représenter seulement l'environnement exécutable observé.
- **Question :** où tournent les artefacts au Sprint 0 ?
- **Périmètre :** navigateur, Vite, Spring Boot et PostgreSQL local.
- **Exclus :** Docker, Nginx, cloud et production.
- **Sources dépôt :** README, configurations Vite/Spring/PostgreSQL.
- **Règles appliquées :** nœuds d'exécution et connexions observables.
- **Hypothèse :** tous les services de développement tournent sur le même poste.

`07-deployment/deployment-target.mmd`

- **Nom :** déploiement cible conditionnel.
- **Objectif :** matérialiser US23 sans inventer un fournisseur cloud.
- **Question :** quelle topologie minimale Docker/Nginx est envisagée ?
- **Périmètre :** hébergement EPT abstrait, proxy, frontend, API, collecteur, DB et SMTP.
- **Exclus :** Kubernetes, AWS/Azure, DNS/TLS exacts, réplication et queue.
- **Sources dépôt :** US23 et architecture DevOps.
- **Règles appliquées :** environnements d'exécution distincts et connexions annotées.
- **Hypothèse :** collecteur déployable séparément ; cible d'hébergement non décidée.

`08-data/erd-target.mmd`

- **Nom :** ERD relationnel cible.
- **Objectif :** préparer les migrations sans confondre tables et objets métier.
- **Question :** quelles clés et cardinalités minimales supporteront les cas d'usage ?
- **Périmètre :** persistance opérationnelle centrale.
- **Exclus :** partitions de métriques, détails rapports, refresh tokens et schéma d'audit complet.
- **Sources dépôt :** modèle initial, ADR-0001, US02/US03/US13/US19.
- **Règles appliquées :** PK/FK explicites, table associative pour le N–N, relations en notation crow's foot.
- **Hypothèses :** clé d'occurrence unique ; site obligatoire ; noms physiques à valider avant migration.

`09-c4/context.mmd` **et** `09-c4/containers.mmd`

- **Nom :** C4 contexte et conteneurs cibles.
- **Objectif :** fournir deux niveaux de zoom pour les parties prenantes et l'équipe.
- **Question :** avec qui le système interagit-il, puis quels conteneurs le réalisent ?
- **Périmètre :** système global et cible applicative.
- **Exclus :** niveau code et component C4, redondants à ce stade.

- **Sources dépôt :** README, ADR-0001, architecture DevOps et backlog.
- **Règles appliquées :** une abstraction par niveau ; personnes/systèmes externes au contexte ; technologies dans les conteneurs.
- **Hypothèses :** worker séparé et WebSocket cible.
- **Support :** syntaxe Mermaid C4 officielle mais expérimentale.

Références par catégorie

- UML : https://www.omg.org/spec/UML/2.5.1/PDF — sémantique normative commune.
- Classes : https://mermaid.js.org/syntax/classDiagram.html — membres, relations et cardinalités.
- Séquences : https://mermaid.js.org/syntax/sequenceDiagram.html — acteurs, participants, boucles et alternatives.
- États : https://mermaid.js.org/syntax/stateDiagram.html — pseudo-états et transitions.
- Activités et cas d'utilisation de substitution : https://mermaid.js.org/syntax/flowchart.html — nœuds, décisions et sous-graphes.
- ERD : https://mermaid.js.org/syntax/entityRelationshipDiagram.html — entités, attributs et crow's foot.
- Architecture : https://mermaid.js.org/syntax/architecture.html — syntaxe disponible depuis Mermaid 11.1.
- C4 : https://mermaid.js.org/syntax/c4.html et https://c4model.com/diagrams — syntaxe expérimentale et niveaux de zoom.

Annexe C — Correspondance modèle persistant et code

Modèle persistant : correspondance avec le diagramme

Le diagramme de référence est persistence-model-uml.mmd. Il est écrit en français ; le code (classes, tables, API) est en anglais. Ce document relie les deux.

Modules

Module Java	Contenu	Dépend de
`security`	`AppUser`, `UserSession`, `PasswordResetToken`	rien
`inventory`	`Site`, `Equipment`, `MonitoredService`, `CollectionConfiguration`, `MetricDefinition`, `CollectionAgent`	`audit` (service)
`monitoring`	`Metric`	`inventory.domain`
`access`	`SiteAccess`, `EquipmentAccess`	`security.domain`, `inventory.domain`
`alerting`	`AlertRule`, `RuleSiteScope`, `RuleEquipmentScope`, `Alert`, `AlertAssignment`, `AlertEvent`, `Maintenance`	`security.domain`, `inventory.domain`
`reporting`	`NotificationPreference`, `Notification`, `NotificationAttempt`, `Report`, `ReportSchedule`, `ScheduleRecipient`, `ScheduleSiteScope`, `ScheduleEquipmentScope`	`security.domain`, `inventory.domain`, `alerting.domain`
`audit`	`AuditEntry`	`security`

Ces dépendances sont vérifiées par ArchUnit (voir archunit.md). `access` est un module à part pour éviter un cycle `security` ↔ `inventory`.

Entités

Diagramme	Classe	Table
Utilisateur	`AppUser`	`app_user`
SessionUtilisateur	`UserSession`	`user_session`
JetonReinitialisation	`PasswordResetToken`	`password_reset_token`
AccesSite	`SiteAccess`	`site_access`
AccesEquipement	`EquipmentAccess`	`equipment_access`

Diagramme	Classe	Table
Site	`Site`	`site`
Equipement	`Equipment`	`equipment`
ServiceSupervise	`MonitoredService`	`monitored_service`
ConfigurationCollecte	`CollectionConfiguration`	`collection_configuration`
DefinitionMetrique	`MetricDefinition`	`metric_definition`
AgentCollecte	`CollectionAgent`	`collection_agent`
Metrique	`Metric`	`metric`
RegleAlerte	`AlertRule`	`alert_rule`
RegleSite	`RuleSiteScope`	`rule_site_scope`
RegleEquipement	`RuleEquipmentScope`	`rule_equipment_scope`
Alerte	`Alert`	`alert`
AffectationAlerte	`AlertAssignment`	`alert_assignment`
EvenementAlerte	`AlertEvent`	`alert_event`
Maintenance	`Maintenance`	`maintenance`
PreferenceNotification	`NotificationPreference`	`notification_preference`
Notification	`Notification`	`notification`
TentativeNotification	`NotificationAttempt`	`notification_attempt`
Rapport	`Report`	`report`
PlanificationRapport	`ReportSchedule`	`report_schedule`
DestinataireRapport	`ScheduleRecipient`	`schedule_recipient`
PlanificationRapportSite	`ScheduleSiteScope`	`schedule_site_scope`
PlanificationRapportEquipement	`ScheduleEquipmentScope`	`schedule_equipment_scope`
EntreeAudit	`AuditEntry`	`audit_entry`

États et énumérations

Diagramme	Énumération	Valeurs
EtatAlerte	`AlertState`	`OPEN` (OUVERTE), `IN_PROGRESS` (PRISE_EN_CHARGE), `RESOLVED` (RESOLUE), `ARCHIVED` (ARCHIVEE), `DELETED` (SUPPRIMEE)
TypeEquipement	`EquipmentType`	`SERVER`, `ROUTER`, `SWITCH`, `VIRTUAL_MACHINE`, `WIFI_ACCESS_POINT` — des **valeurs**, pas des sous-classes

Diagramme	Énumération	Valeurs
StatutEquipement	`EquipmentStatus`	`UP`, `DOWN`, `DEGRADED`, `MAINTENANCE`, `UNKNOWN`
StatutSite	`SiteStatus`	`ACTIVE`, `INACTIVE`
Severite	`Severity`	`INFO`, `WARNING`, `CRITICAL`
TypeAlerte	`AlertType`	`THRESHOLD` (SEUIL), `COLLECTION` (COLLECTE)
TypeResolution	`ResolutionType`	`MANUAL`, `AUTOMATIC`
EtatMaintenance	`MaintenanceState`	`PLANNED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`
ModeCollecte	`CollectionMode`	`AGENT`, `SNMP`
VersionSnmp	`SnmpVersion`	`V2C`, `V3`
StatutAgent	`AgentStatus`	`PENDING`, `ACTIVE`, `INACTIVE`, `REVOKED`
QualiteMesure	`MeasurementQuality`	`GOOD`, `DEGRADED`, `BAD`
RoleUtilisateur	`Role`	`ADMIN`, `NOC_OPERATOR`, `VIEWER`

Les autres énumérations (`ServiceType`, `ValueType`, `ComparisonOperator`, `AlertEventType`, `NotificationChannel`, `NotificationState`, `SendResult`, `ReportType`, `ReportState`, `ReportFormat`, `ReportFrequency`) sont dans le package `domain` de leur module. Chacune est stockée en texte et protégée par une contrainte `CHECK` en base.

Écarts assumés avec le diagramme

- **Noms en anglais** (décision d'équipe) : la table ci-dessus fait foi.
- `Equipment.inventoryCode` et `Equipment.fictitious` n'existent pas dans le diagramme : ils viennent de l'API déjà livrée (code d'inventaire unique, données de démonstration).
- `Metric.key` est conservé en plus de la relation vers `MetricDefinition` (facultative) pour les mesures déjà enregistrées.
- `AuditEntry.username` copie le nom du compte à l'instant de l'action ; la relation `author` peut devenir vide si le compte est supprimé.
- Les durées (`Duration`) sont stockées en secondes (`*_seconds`, `BIGINT`) pour rester indépendantes du moteur.

Où chaque règle est garantie

Règle du diagramme	Garantie par
Une maintenance cible exactement un site ou un équipement	`CHECK ck_maintenance_exactly_one_target`
Une notification vient d'une alerte ou d'un rapport, jamais des deux	`CHECK ck_notification_exactly_one_origin`
`THRESHOLD` exige une règle, `COLLECTION` n'en a pas	`CHECK ck_alert_type_rule_consistency`

Règle du diagramme	Garantie par
Mode `SNMP` exige une version ; mode `AGENT` sans paramètre SNMP	`CHECK` sur `collection_configuration`
Au plus un agent actif par équipement	index unique partiel `uq_collection_agent_one_active` (PostgreSQL)
Une seule alerte active par couple équipement-règle, ou par équipement pour `COLLECTION`	index uniques partiels `uq_alert_one_active_*` (PostgreSQL)
Au plus une affectation active par alerte	index unique partiel `uq_alert_assignment_one_active` (PostgreSQL)
Une préférence de notification par utilisateur	contrainte `UNIQUE (user_id)`
La portée d'une planification contient au moins un site ou un équipement	**service** (non exprimable en contrainte portant sur une seule table)
Compositions (sessions, services, configurations, événements, tentatives…)	clés étrangères `ON DELETE CASCADE`

Secrets de collecte

`CollectionConfiguration` ne stocke que des **références** vers un gestionnaire de secrets (`communityReference` , `securityIdentity` , `secretReference`), jamais les secrets eux-mêmes. Ces champs portent `@JsonIgnore` , aucune entité ne redéfinit `toString()` et les contrôleurs n'exposent aucune entité (règle ArchUnit). Ces propriétés sont testées dans `PersistenceModelTests` .

Migrations

- `db/migration/V7__persistent_model.sql` : tables, clés, `CHECK` et index, plus l'évolution additive des tables existantes (aucune donnée supprimée).
- `db/migration/V8__partial_unique_indexes.sql` : index uniques partiels (PostgreSQL). Depuis l'ADR-0003, un seul dossier de migrations est appliqué en développement, en test (Testcontainers) et en production.
- V9 partitionne la table `metric` par mois (ADR-0004) ; V10 ajoute `family_id` et `replaced_by_id` aux sessions (ADR-0005) ; V11 ajoute `alert_rule.recovery_threshold` (hystérésis) ; V12 ajoute `metric_hourly` et la fonction de purge des partitions expirées. V13 ajoute `collection_agent.token_hash` . La prochaine migration est la V14.